



**Universidad
Europea**

UNIVERSIDAD EUROPEA DE ANDALUCÍA

ESCUELA POLITÉCNICA

**MÁSTER UNIVERSITARIO EN ANÁLISIS DE GRANDES
VOLÚMENES DE DATOS (BIG DATA)**

TRABAJO FIN DE MÁSTER

**DESARROLLO DE UN SISTEMA
ESCALABLE DE MICROSERVICIOS PARA EL
PROCESAMIENTO Y ANÁLISIS
OPERACIONAL DE DATOS IOT**

BORIS RENEE PATERNINA PEREZ

Dirigido por

VICTOR GOMEZ GUIRADO

CURSO 2025 - 2026

TÍTULO: DESARROLLO DE UN SISTEMA ESCALABLE DE MICROSERVICIOS PARA EL
PROCESAMIENTO Y ANÁLISIS OPERACIONAL DE DATOS IOT

AUTOR: BORIS RENEE PATERNINA PEREZ

TITULACIÓN: MÁSTER UNIVERSITARIO EN ANÁLISIS DE GRANDES VOLÚMENES DE
DATOS (BIG DATA)

DIRECTOR DEL PROYECTO: VICTOR GOMEZ GUIRADO

FECHA: Septiembre de 2026

RESUMEN

Las referencias disponibles sobre arquitecturas Kappa aplicadas al Internet de las Cosas (IoT) tienen mayoritariamente un carácter demostrativo y dependen en algún punto de servicios gestionados o plataformas propietarias; ninguna implementación de código abierto revisada integra a la vez un puente MQTT-Kafka propio, la gobernanza de esquemas mediante Avro y Apicurio, y una estrategia de doble sumidero que separe el consumo operacional del analítico. Este trabajo aborda esa carencia con el diseño, el desarrollo y la validación de un sistema escalable de microservicios, containerizado y basado en herramientas de código abierto, que ingiere, gobierna, procesa y visualiza telemetría de consumo energético de edificios en tiempo real. La solución implementa un simulador de dispositivos, un broker MQTT ligero con un microservicio de bridging hacia Kafka, la validación de cada evento contra un esquema Avro registrado, el procesamiento en flujo con Spark Structured Streaming y agregación por ventanas, y un doble sumidero hacia TimescaleDB para la explotación operacional en Grafana y hacia PostgreSQL para el análisis en Power BI. Cinco objetivos, cada uno con un indicador clave de rendimiento verificable, se cumplieron dentro de los umbrales fijados, incluida la recuperación ante el fallo de cualquier servicio sin pérdida de datos. El resultado queda disponible como implementación de referencia pública y reproducible.

Palabras clave: arquitectura Kappa, Internet de las Cosas, procesamiento de flujos, gobernanza de esquemas, Apache Kafka, código abierto

ABSTRACT

Most available references on Kappa architectures applied to the Internet of Things (IoT) are largely demonstrative and, at some point in the chain, depend on managed services or proprietary platforms; none of the reviewed open-source implementations combines a purpose-built MQTT-to-Kafka bridge, schema governance through Avro and Apicurio, and a dual-sink strategy that separates operational from analytical consumption. This work addresses that gap by designing, developing and validating a scalable microservices system, containerized and based on open-source tools, that ingests, governs, processes and visualizes building energy-consumption telemetry in real time. The solution implements a device simulator, a lightweight MQTT broker with a bridging microservice to Kafka, validation of every event against a registered Avro schema, stream processing with Spark Structured Streaming and windowed aggregation, and a dual sink to TimescaleDB for operational exploitation in Grafana and to PostgreSQL for analysis in Power BI. Five objectives, each with a verifiable key performance indicator, were met within the defined thresholds, including recovery from the failure of any service without data loss. The outcome is available as a public, reproducible reference implementation.

Keywords: Kappa architecture, Internet of Things, stream processing, schema governance, Apache Kafka, open source

Dedicado a mis padres

Índice general

1. RESUMEN DEL PROYECTO	10
1.1. Contexto y justificación	10
1.2. Planteamiento del problema	10
1.3. Objetivos del proyecto	10
1.4. Resultados obtenidos	11
1.5. Estructura de la memoria	11
2. ANTECEDENTES Y ESTADO DEL ARTE	12
2.1. Antecedentes	12
2.1.1. El crecimiento exponencial de los datos IoT y el Big Data	12
2.1.2. Arquitecturas de referencia para el procesamiento por flujos: Lambda y Kappa	12
2.2. Estado del arte	14
2.2.1. Ecosistema de herramientas de código abierto en el ámbito de los datos IoT	14
2.2.2. Retos abiertos: seguridad, interoperabilidad y especialización técnica	16
2.2.3. Criterios de selección de herramientas de procesamiento de flujos	16
2.3. Contexto y justificación	17
2.4. Planteamiento del problema	18
3. OBJETIVOS	19
3.1. Alcance del proyecto	19
3.2. Tareas a desarrollar	19
3.3. Metodología de desarrollo	19
3.3.1. Objetivo 1: Garantizar la persistencia de telemetría rápida y fiable	20
3.3.2. Objetivo 2: Garantizar la gobernanza del esquema de datos	20
3.3.3. Objetivo 3: Procesar y enriquecer los datos en flujo con baja latencia	20
3.3.4. Objetivo 4: Ofrecer visualización diferenciada operacional y analítica	20
3.3.5. Objetivo 5: Validar la resiliencia del sistema	20
3.4. Beneficios del proyecto	21
4. ÉTICA Y COMPROMISO SOCIAL	22
4.1. Igualdad de género	22
4.2. Objetivos de Desarrollo Sostenible	22
5. DESARROLLO DEL PROYECTO	24
5.1. Planificación del proyecto	24
5.2. Descripción de la solución, metodologías y herramientas empleadas	24
5.2.1. Visión general de la arquitectura	24
5.2.2. Fuente de datos utilizada	26

5.2.3. Criterios de selección de herramientas	27
5.2.4. Estructura del repositorio	28
5.2.5. Preparación de los datos	29
5.2.6. Simulador de telemetría	29
5.2.7. Broker MQTT (Eclipse Mosquitto)	30
5.2.8. Bridge MQTT → Kafka	31
5.2.9. Registro de esquemas (Apicurio Registry)	32
5.2.10. Apache Kafka	34
5.2.11. Apache Spark Structured Streaming	35
5.2.12. TimescaleDB	35
5.2.13. PostgreSQL	37
5.2.14. Grafana	38
5.2.15. Power BI	39
5.3. Recursos requeridos	40
5.4. Presupuesto	41
5.5. Viabilidad	41
5.5.1. Viabilidad técnica	41
5.5.2. Viabilidad económica	41
5.5.3. Sostenibilidad	42
5.6. Resultados del proyecto	42
5.6.1. Objetivo 1: Garantizar la persistencia de telemetría rápida y fiable	42
5.6.2. Objetivo 2: Garantizar la gobernanza del esquema de datos	43
5.6.3. Objetivo 3: Procesar y enriquecer los datos en flujo con baja latencia	44
5.6.4. Objetivo 4: Ofrecer visualización diferenciada operacional y analítica	44
5.6.5. Objetivo 5: Validar la resiliencia del sistema	47
6. EJECUCIÓN Y VALIDACIÓN DEL SISTEMA	49
6.1. Requisitos previos	49
6.2. Modo automático de ejecución	49
6.3. Arranque manual (emisión de KPIs)	50
6.4. Escalado horizontal del bridge	50
6.5. Herramientas de validación	51
6.5.1. Inyección de errores durante la publicación de eventos MQTT	51
6.5.2. Visualización de los eventos rechazados por el bridge	52
7. DISCUSIÓN	53
7.1. Limitaciones del proyecto	53
7.2. Impacto del resultado	53
8. CONCLUSIONES	55
8.1. Conclusiones del trabajo	55
9. FUTURAS LÍNEAS DE TRABAJO	56
Bibliografía	57

Índice de Figuras

2.1. Arquitectura Lambda	13
2.2. Arquitectura Kappa	13
5.1. Arquitectura del pipeline IoT	25
5.2. Dashboard 'Estado del pipeline'.	45
5.3. Dashboard 'Consumo energético'.	45
5.4. Dashboard 'Calidad y anomalías'.	46
5.5. Página 'Consumo de Energía'.	46
5.6. Página 'Eficiencia Eléctrica'.	47
5.7. Página 'Detección de Anomalías'.	47

Índice de Tablas

2.1. Principales brokers MQTT con licencia aprobada por la Open Source Initiative	14
2.2. Principales plataformas de mensajería persistente con licencia aprobada por la Open Source Initiative	15
5.1. Cronograma y esfuerzo de las actividades del proyecto	24
5.2. Columnas de la tabla de hechos (ashrae_telemetry.parquet).	26
5.3. Columnas de la tabla de dimensión (ashrae_buildings.parquet).	27
5.4. Columnas de la línea base por sensor (ashrae_sensor_baseline.parquet).	27
5.5. Relación entre el factor de aceleración y el ritmo de publicación.	29
5.6. Campos del mensaje JSON publicado por el simulador.	30
5.7. Columnas de telemetry_metrics.	36
5.8. Columnas de streaming_progress.	37
5.9. Columnas de telemetry_events.	38
5.10. Paneles del dashboard 'Estado del pipeline'.	38
5.11. Paneles del dashboard 'Consumo energético'.	39
5.12. Paneles del dashboard 'Calidad y anomalías'.	39
5.13. Costes del proyecto.	41
5.14. Resultado del Objetivo 1: persistencia de telemetría rápida y fiable.	42
5.15. Resultado del Objetivo 2: gobernanza del esquema de datos.	43
5.16. Resultado del Objetivo 3: latencia de micro-lotes de Spark.	44
5.17. Resultado del Objetivo 4: configuración de los dashboards de Grafana.	44
5.18. Resultado del Objetivo 5: pruebas de resiliencia del sistema.	48
6.1. Herramientas de validación de KPIs.	51

Capítulo 1. RESUMEN DEL PROYECTO

1.1 Contexto y justificación

Junto al aumento exponencial de sensores y dispositivos conectados a internet, existen retos lejos de ser solventados causados por la gran variedad de protocolos, formatos y estándares de los fabricantes de estos dispositivos. El volumen de datos generado por el Internet de las Cosas (IoT) impone requisitos que deben ser satisfechos por arquitecturas capaces de procesar y analizar datos en tiempo real. Sobre la arquitectura Kappa se encuentran multitud de publicaciones, tutoriales y documentación de todo calibre que no llegan a ofrecer una implementación extremo a extremo de esta arquitectura para procesar y analizar datos por flujos. La mayoría de los recursos formativos y de referencia dependen en algún punto de servicios gestionados o plataformas propietarias, dificultando una experiencia de aprendizaje completa basada en herramientas de código abierto.

1.2 Planteamiento del problema

El problema que aborda el presente proyecto es la falta de una implementación de referencia, de código abierto y containerizada, de una arquitectura Kappa para IoT que integre bridging MQTT-Kafka, gobernanza de esquemas mediante Avro/Apicurio, y una estrategia de doble sumidero para consumo operacional y analítico. Por tanto, se propone contribuir a llenar vacíos de información detectados durante la investigación del estado del arte sobre arquitecturas públicas y de código abierto con un proyecto que tiene carácter académico y científico-técnico; no responde a un encargo empresarial y su aporte de innovación está en integrar en una sola arquitectura todos los elementos antes mencionados.

1.3 Objetivos del proyecto

El objetivo general es diseñar, desarrollar y validar un sistema escalable de microservicios para la ingesta, el procesamiento y el análisis operacional de datos IoT. Para cubrir ese alcance se plantean las tareas de analizar el estado del arte y las herramientas de código abierto disponibles, implementar un simulador de dispositivos IoT, implementar la arquitectura de microservicios containerizada; con servicios de ingesta sobre protocolo estándar, de gobernanza y validación de esquemas, de agregación y transformación, y de almacenamiento para series temporales y para consumo analítico, e implementar los servicios de visualización interactiva. Cada tarea se concreta en uno de cinco objetivos: persistencia de telemetría rápida y fiable, gobernanza del esquema de datos, procesamiento en flujo con baja latencia, visualización diferenciada operacional y analítica, y resiliencia del sistema, cada uno acompañado de un indicador clave de rendimiento (KPI) verificable.

1.4 Resultados obtenidos

Los cinco objetivos se cumplieron dentro de los umbrales fijados por sus KPIs. La telemetría validada se persiste dentro del tiempo objetivo y con una tasa de pérdida nula; la totalidad de los eventos ingeridos se valida contra el esquema Avro registrado sin fallos de deserialización; cada micro-lote de procesamiento se resuelve por debajo del límite de latencia sin incremento registrado; la visualización se ofrece mediante tres *dashboards* de Grafana y tres páginas de Power BI que cubren el consumo energético, la eficiencia operativa, la detección de anomalías y el análisis histórico; y el sistema se recupera de la caída de sus servicios por debajo del tiempo objetivo y sin pérdida de datos. El resultado es la implementación completa, containerizada y reproducible de una arquitectura Kappa para IoT construida con herramientas de código abierto, disponible para su adopción y adaptación en proyectos similares de monitorización energética.

1.5 Estructura de la memoria

Antecedentes y estado del arte sitúa el crecimiento de los datos IoT, compara las arquitecturas Lambda y Kappa, revisa el ecosistema de herramientas de código abierto por capas y algunos retos abiertos, y formula el planteamiento del problema. **Objetivos** define el alcance, las tareas a desarrollar y los objetivos con sus KPIs y acciones. **Ética y compromiso social** recoge la relación del trabajo con la igualdad de género y con los Objetivos de Desarrollo Sostenible. **Desarrollo del proyecto** describe la planificación, la arquitectura y las herramientas empleadas, la preparación de los datos, cada componente del *pipeline*, los recursos y el presupuesto, y presenta los resultados obtenidos para cada objetivo. **Ejecución y validación del sistema** documenta los modos de arrancar el *pipeline* y las herramientas para medir KPIs y ejecutar pruebas de fallos. **Discusión** expone las limitaciones del proyecto y el impacto de los resultados. **Conclusiones** valora el cumplimiento de los objetivos y sus KPIs respectivos. **Futuras líneas de trabajo** plantea la evolución de la orquestación del sistema. Se culmina la memoria con la **bibliografía** y los **anexos**.

Capítulo 2. ANTECEDENTES Y ESTADO DEL ARTE

2.1 Antecedentes

2.1.1. El crecimiento exponencial de los datos IoT y el Big Data

El rápido crecimiento en el número de dispositivos conectados al Internet de las Cosas (IoT) y el aumento exponencial en el uso de sus datos sugieren claramente que el desarrollo del Big Data está estrechamente vinculado con el IoT. La explosión en el volumen, velocidad y variedad de los datos generados por miles de millones de dispositivos ha superado la capacidad de los enfoques de procesamiento tradicionales, obligando a la creación y adopción de arquitecturas capaces de ingerir, procesar y almacenar flujos continuos de datos en tiempo real (Y. Liu [1]).

Junto a los factores tecnológicos impulsados por el crecimiento de los datos IoT, debe considerarse que el valor de éstos depende de la latencia con la que se procesan. Por ejemplo, los datos de temperatura de un motor industrial pierden gran parte de su utilidad inicial si se procesan horas después de haber sido generados, cuando la ventana para prevenir el evento adverso ya se ha cerrado. Este factor temporal motiva la centralidad que ha adquirido el procesamiento por flujos (*stream processing*) versus el procesamiento por lotes clásico para los pipelines IoT modernos.

2.1.2. Arquitecturas de referencia para el procesamiento por flujos: Lambda y Kappa

En la literatura sobre procesamiento de grandes volúmenes de datos en flujo destacan dos patrones arquitectónicos ampliamente documentados. La arquitectura Lambda fue propuesta originalmente por N. Marz [2] en 2011 (posteriormente detallada junto con J. Warren [3]), que parte de la premisa de que toda consulta puede expresarse como una función sobre la totalidad de los datos ($query = function(all\ data)$), pero calcular esa función *en tiempo real* sobre el dataset completo resulta prohibitivamente costoso. Para resolver esto, se implementa una capa de procesamiento por lotes (que recalcula vistas precisas sobre el histórico completo), junto a una capa de velocidad (que ofrece vistas aproximadas de baja latencia sobre los datos más recientes), que después se fusionan en una capa de servicio.

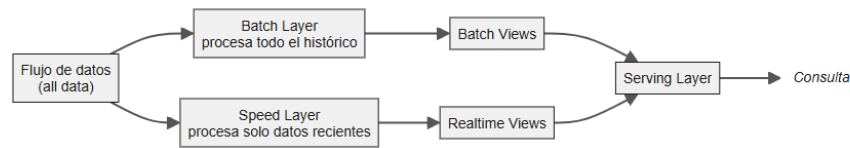


Figura 2.1. Arquitectura Lambda: las capas por lote y por flujo se fusionan en la capa de servicio. Elaboración propia a partir de [2], [3].

No obstante la amplia adopción de la arquitectura Lambda, J. Kreps [4] señala que esta exige implementar y mantener la misma lógica de transformación por duplicado: una en el sistema por lote y otra en el sistema por flujo, y sincronizar ambas transformaciones resulta muy costoso operativamente. H. Cao y M. Wachowicz [5] añaden que la dependencia de infraestructura de cómputo intensivo hace esta arquitectura poco adecuada para muchos escenarios IoT.

Como alternativa, J. Kreps [4] propuso en 2014 la arquitectura Kappa, que elimina la capa por lotes y utiliza un flujo reproducible (*replayable stream*) como fuente de verdad única. Cuando hace falta reprocesar el flujo (por un cambio de lógica o corrección de un bug), se lanza una nueva instancia de job de flujo (con una versión mejorada del código, corriendo sobre el mismo framework) que se ejecuta reproduciendo el flujo desde el principio, escribiendo su resultado en una nueva tabla de salida y en paralelo a la tabla que ya está en producción; una vez que el nuevo job se sincroniza al presente, la aplicación se redirige a leer de la tabla nueva, y solo entonces se puede retirar la tabla y el job antiguos [4].

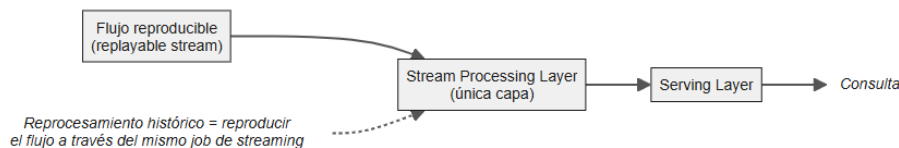


Figura 2.2. Arquitectura Kappa: una única capa de flujo, con reprocesamiento histórico mediante reproducción del flujo. Elaboración propia a partir de [4].

Ejemplos recientes de implementación práctica de Kappa en contextos IoT incluyen a S. Shinde [6], quien construye un pipeline de monitorización de temperatura y detección de anomalías con Kafka, Spark Streaming y Streamlit, y a G. Zefkilis [7], que documenta una variante con Kafka, Spark Streaming, StarRocks y MinIO sobre Docker. Cabe señalar, no obstante, que ambas citas son tutoriales técnicos de un único autor.

2.2 Estado del arte

2.2.1. Ecosistema de herramientas de código abierto en el ámbito de los datos IoT

El ecosistema de herramientas de código abierto disponibles para cada capa del pipeline ha madurado considerablemente, como ilustran las implementaciones de S. Shinde [6] y G. Zefkilis [7] comentadas en la sección anterior.

En la capa de ingesta tenemos varios estándares disponibles para los protocolos de comunicación con los dispositivos IoT; entre ellos **MQTT** (ISO/IEC 20922) ofrece un modelo de publicación-suscripción por tópicos con broker central y niveles de calidad de servicio configurables, adecuado para dispositivos numerosos y de conectividad intermitente; **CoAP** (RFC 7252) sigue un modelo de petición-respuesta sobre UDP, pensado para dispositivos de recursos muy limitados, y solo incorpora publicación-suscripción mediante extensiones; **AMQP 1.0** (ISO/IEC 19464) está orientado a mensajería empresarial con mayores garantías y coste de implementación; y **DDS** (especificación OMG) adopta un modelo descentralizado sin broker, propio de sistemas de control en tiempo real.

Estableciendo MQTT como protocolo para este proyecto, los brokers disponibles para la recolección de telemetría se resumen en la Tabla 2.1.

Broker	Licencia	Lenguaje	Observaciones
Eclipse Mosquitto	EPL 2.0 / EDL 1.0	C	Implementación de referencia de la Eclipse Foundation; exclusivamente MQTT, de huella mínima
HiveMQ Community Ed.	Apache 2.0	Java	La extensión de integración con Kafka es producto comercial
VerneMQ	Apache 2.0	Erlang	Orientado a despliegues distribuidos, con clustering nativo
NanoMQ	MIT	C	Diseñado para el borde de la red
Apache ActiveMQ Artemis	Apache 2.0	Java	Multiprotocolo: AMQP 1.0, MQTT 3.1.1 y 5.0, y STOMP
RabbitMQ	MPL 2.0	Erlang	Broker de colas con soporte de MQTT 3.1, 3.1.1 y 5.0

Tabla 2.1. Principales brokers MQTT con licencia aprobada por la Open Source Initiative

De este inventario conviene excluir EMQX [8], habitualmente citado entre las opciones de licencia libre: desde su versión 5.9.0 el proyecto abandonó la licencia Apache 2.0 en favor de una licencia *source-available* no aprobada por la Open Source Initiative.

Para el acoplamiento entre los brokers de recolección y las plataformas de mensajería persistente, no existe una solución de código abierto integrada en el propio broker: los que ofrecen un puente nativo lo hacen desde una edición comercial [9] o bajo licencias *source-available* [8]. En consecuencia, toda arquitectura restringida a herramientas de código abierto debe incorporar un componente adicional de *bridging*. Este puede ser un conector genérico, como el conector MQTT de Stream Reactor para Kafka Connect [10], o un conector desarrollado a la medida si se quiere asignar funcionalidades adicionales a dicho componente.

Para la capa de almacenamiento reproducible del flujo que alimentará a la capa de procesamiento, la oferta de plataformas de mensajería persistente es igualmente amplia, como recoge la Tabla 2.2.

Plataforma	Licencia	Lenguaje	Observaciones
Apache Kafka	Apache 2.0	Java/Scala	Registro particionado con retención configurable y desplazamiento por consumidor
Apache Pulsar	Apache 2.0	Java	Separa cómputo y almacenamiento mediante BookKeeper; multi-tenencia y geo-replicación nativas
Apache RocketMQ	Apache 2.0	Java	Reproducción de mensajes por tiempo o por desplazamiento; ecosistema concentrado en Asia
NATS (JetStream)	Apache 2.0	Go	Binario único sin dependencia de JVM; ecosistema de conectores más reducido
RabbitMQ Streams	MPL 2.0	Erlang	Registro persistente añadido sobre un broker de colas

Tabla 2.2. Principales plataformas de mensajería persistente con licencia aprobada por la Open Source Initiative

Todas las plataformas de la Tabla 2.2 son técnicamente viables para una arquitectura Kappa, puesto que ofrecen registro persistente, reproducción por desplazamiento y consumidores duraderos; requisitos fundamentales que exige todo diseño.

En la capa de procesamiento de flujos, Apache Flink se ha consolidado como la mejor opción para el cómputo en tiempo real usando la semántica de eventos, mientras que Kafka Streams ocupa un nicho más acotado como librería embebida para aplicaciones Java/Scala nativas de Apache Kafka que no requieren un clúster de procesamiento independiente. Apache Spark Structured Streaming, por su parte, se presenta como el estándar de la industria para el procesamiento por micro-lotes, apoyado en la amplitud de su ecosistema y la unificación del procesamiento en lotes y en flujo bajo una misma API; característica que comparte Apache Flink pero que en Spark se beneficia de una integración más amplia con el ecosistema de almacenamiento *lakehouse* y herramientas SQL (S. Lakshminpathy [11]).

Para el almacenamiento de series temporales de los datos IoT, sistemas especializados como InfluxDB [12], TimescaleDB [13] y QuestDB [14] compiten por ofrecer el mejor equilibrio entre velocidad de escritura, compresión de datos y capacidad de consulta analítica; sin embargo conviene advertir que buena parte de los benchmarks de rendimiento publicados provienen directamente de los creadores de cada producto como se aprecia en las citadas fuentes, por lo que dichos benchmarks deben interpretarse con cautela hasta que existan estudios independientes.

En la capa de visualización, Grafana se ha convertido en el estándar para paneles operacionales sobre datos de series temporales, en muchos casos combinado con Prometheus para monitoreo de infraestructura.

T. Domínguez-Bolaño et al. [15] complementan la panorámica anterior con un análisis comparativo de plataformas IoT de código abierto de propósito más amplio (como ThingsBoard,

openHAB, OpenRemote o SiteWhere) que integran varias de estas capas en un único producto, frente al enfoque modular de ensamblar componentes especializados por capa que caracteriza a los pipelines contruidos a medida. Ambos enfoques (plataforma integrada y ensamblaje modular de componentes) representan compromisos distintos entre rapidez de despliegue y flexibilidad de adaptación ante los requisitos específicos de un proyecto.

En conjunto, la evidencia revisada sugiere que las herramientas de código abierto en el ámbito de los datos IoT son hoy suficientemente maduras para competir con las ofertas propietarias en la nube en la mayoría de los casos de uso, a costa de una mayor carga operativa que exige experiencia técnica para configurar, escalar y asegurar cada componente del pipeline.

2.2.2. Retos abiertos: seguridad, interoperabilidad y especialización técnica

Pese a la madurez alcanzada por las herramientas disponibles, Y. Liu [1] y T. Domínguez-Bolaño et al. [15] plantean, de forma independiente, tres retos que permanecen abiertos:

- En primer lugar, la seguridad: la superficie de ataque se amplía con cada dispositivo IoT conectado, muchos de los cuales carecen de mecanismos de seguridad inherentes debido a la naturaleza distribuida de las redes IoT y a la ausencia de protocolos de seguridad estandarizados entre fabricantes [1].
- En segundo lugar, la interoperabilidad: la coexistencia de múltiples protocolos, formatos de datos y estándares entre fabricantes sigue dificultando la integración de dispositivos heterogéneos dentro de una misma plataforma [15].
- En tercer lugar, la especialización técnica requerida: la complejidad y la escala de los datos generados por los dispositivos IoT hacen necesario el uso de herramientas o plataformas especializadas para procesarlos de forma efectiva [1]. El manejo de estas herramientas puede exigir un nivel de dominio técnico que no siempre está disponible en equipos de tamaño reducido.

2.2.3. Criterios de selección de herramientas de procesamiento de flujos

La elección de un sistema concreto para el procesamiento de flujos dentro de una arquitectura Kappa no tiene una respuesta universal: distintas herramientas optimizan frente a distintos compromisos, y la decisión debe fundamentarse en los criterios aplicables al proyecto de procesamiento de flujo, no en percepciones públicas o popularidad de una herramienta.

A continuación se plantean algunos criterios considerados y su aplicación al dominio del procesamiento de flujo en general:

- **Requisitos de latencia vs. cadencia de generación de datos:** si la frecuencia de emisión de los dispositivos es del orden de segundos o más lenta, un motor de procesamiento por micro-lotes no introduce una degradación perceptible de la información generada; ya que la ventaja de latencia sub-segundo con un motor de flujo nativo solo es relevante cuando el caso de uso exige control reactivo o detección de eventos críticos en tiempo real.

- **Madurez del ecosistema y de las APIs en el lenguaje de implementación:** la profundidad de soporte de un lenguaje concreto (Java, Scala, Python) varía entre motores; un ecosistema con API nativa más madura reduce fricción de desarrollo y mantenimiento.
- **Unificación del modelo de procesamiento *batch/streaming*:** algunos motores permiten expresar lógica de procesamiento por lotes y en flujo bajo una misma API, lo que simplifica la arquitectura cuando ambos modos de procesamiento coexisten en el sistema.
- **Disponibilidad de talento y curva de adopción:** la disponibilidad de profesionales con experiencia previa en una tecnología concreta condiciona la viabilidad de mantenimiento y escalado de un sistema en producción a largo plazo.

Específicamente, en el dominio del procesamiento de flujo de dispositivos IoT industriales, la cadencia habitual de generación de eventos (normalmente en la escala de segundos) sitúa las exigencias de latencia del sistema por debajo del umbral en el que las ventajas de un motor de flujo nativo resultan necesarias. Por tanto, un motor de procesamiento por micro-lotes con ventanas de agregación temporal ofrece garantías de tolerancia a fallos y semántica *exactly-once* equivalentes a las de un motor de flujo nativo, sin incurrir en complejidades operativas adicionales. Este análisis no excluye la idoneidad de los motores de flujo nativos en escenarios de mantenimiento predictivo, sistemas de control reactivo o detección de eventos críticos en tiempo real, donde la latencia resulta un factor determinante del diseño.

2.3 Contexto y justificación

El estado del arte revisado en la sección anterior pone de manifiesto un patrón recurrente: la mayor parte de las referencias sobre arquitecturas Kappa para IoT disponibles hoy (tutoriales de ingeniería, publicaciones de blog técnico, documentación de proyectos de código abierto) tienen un carácter fundamentalmente demostrativo. Su objetivo es ilustrar un concepto (por ejemplo, monitoreo de temperatura con Kafka, Spark Streaming y Streamlit, o un pipeline equivalente con StarRocks y MinIO), más que ofrecer una implementación completa y reproducible. Esta brecha entre la abundancia de casos ilustrativos y la escasez de ejemplos reproducibles constituye la primera motivación de este proyecto: construir, documentar y poner a disposición pública un pipeline IoT completo, containerizado y basado en herramientas de código abierto, que sirva como trabajo académico evaluable y referencia técnica reutilizable por terceros.

El proceso de revisión de la literatura evidenció vacíos de información que este proyecto puede contribuir a complementar, al menos parcialmente, desde la práctica: no se encontró un trabajo independiente que documente el comportamiento de un broker MQTT de perfil ligero (como Eclipse Mosquitto) acoplado en la capa de ingesta a un microservicio de *bridging* dedicado hacia Kafka; combinación de elementos que la bibliografía consultada menciona de forma tangencial y sin evaluación.

La gobernanza de esquemas mediante Avro y un registro como Apicurio está documentada como buena práctica en arquitecturas de flujo de propósito general (en particular M. Kleppmann [16] describe el fundamento conceptual del uso de esquemas del lector y del escritor

en la codificación de los datos), pero no aparece integrada como componente explícito en ninguna de las implementaciones Kappa-IoT de código abierto revisadas ([6], [7]) ni en los trabajos académicos consultados sobre analítica y arquitectura IoT ([1], [5], [15]). Incorporar esta gobernanza aporta un elemento diferenciador respecto a las implementaciones revisadas, permitiendo la evolución controlada del esquema de telemetría a medida que cambian los tipos de sensor o los campos capturados, y acercando este pipeline a las prácticas ya consolidadas en arquitecturas de flujo de propósito general.

El diseño de doble sumidero responde a una necesidad práctica de separar el consumo operacional en tiempo casi real (monitorización de planta, alertas) del consumo analítico orientado a negocio (reporting, análisis histórico, cruces con otras fuentes corporativas). Este patrón de doble sumidero está documentado en el sector de servicios gestionados propietarios (por ejemplo, el caso de AWS documentado por D. Kim et al. [17]) pero no aparece implementado en ninguna de las arquitecturas Kappa de código abierto revisadas, las cuales asumen un único sumidero de salida.

En conjunto, estos elementos (implementación completa y reproducible, gobernanza de esquemas mediante Avro/Apicurio, y separación de sumideros operacional/analítico) justifican la elección de la arquitectura desarrollada en este proyecto no como una simple aplicación de una solución ya estudiada, sino como una propuesta que extiende la arquitectura Kappa hacia una dirección que la literatura actual documenta de forma incompleta.

2.4 Planteamiento del problema

De la revisión anterior se desprende que, si bien existen múltiples implementaciones ilustrativas de arquitecturas Kappa aplicadas a IoT, y sistemas equivalentes de gobernanza de esquemas y doble sumidero documentados en contextos de *streaming* de propósito general, ninguna implementación Kappa-IoT de código abierto revisada combina simultáneamente (i) un broker MQTT ligero acoplado a un microservicio de bridging propio hacia Kafka; (ii) un registro de esquemas Avro que valide el formato de cada evento transmitido y su admisibilidad entre la capa de ingesta y la capa de procesamiento; y (iii) una estrategia de doble sumidero que separe el consumo de datos operacionales del consumo analítico.

La mayoría de los recursos formativos y de referencia disponibles para diseñar este tipo de arquitecturas dependen, en algún punto de la cadena, de servicios gestionados o plataformas propietarias (colas de mensajería en la nube, motores de flujo como servicio, o plataformas de ingesta IoT gestionadas como AWS IoT Core o Azure IoT Hub), lo que dificulta el diseño en escenarios que requieren control total sobre el despliegue, la no dependencia de un proveedor concreto, o la posibilidad de ejecutar el sistema en un entorno local (o on-premise) mediante microservicios.

Considerando lo anterior, el problema planteado en este proyecto es *la falta de una implementación de referencia, de código abierto y containerizada, de una arquitectura Kappa para IoT que integre bridging MQTT-Kafka, gobernanza de esquemas mediante Avro/Apicurio, y una estrategia de doble sumidero que separe el consumo operacional del analítico.*

Capítulo 3. OBJETIVOS

3.1 Alcance del proyecto

Este proyecto abarca el diseño, desarrollo y validación de un sistema escalable de microservicios para la ingesta, procesamiento, y análisis operacional de datos de dispositivos IoT, utilizando como caso de uso la supervisión y análisis en tiempo real de la telemetría de consumo energético en edificios con medidores de electricidad, agua fría y agua caliente.

3.2 Tareas a desarrollar

Para cubrir el alcance descrito, se plantean las siguientes tareas:

- Analizar el estado del arte en las tecnologías, estrategias y retos actuales en el procesamiento y análisis de datos en flujo.
- Analizar las herramientas de código abierto disponibles para el procesamiento y análisis de datos de dispositivos IoT.
- Diseñar e implementar un simulador de dispositivos IoT para la transmisión de datos de telemetría.
- Diseñar e implementar una arquitectura basada en contenedores para la orquestación y ejecución de los siguientes microservicios:
 - Servicio para la ingesta de datos que acepte protocolos de telemetría estándar.
 - Servicio de gobernanza y validación de esquemas de datos en flujo.
 - Servicios de agregado y transformación que permitan calcular índices de eficiencia energética, efectuar análisis operativo e histórico y hacer detección de anomalías.
 - Servicio de almacenamiento optimizado para series temporales.
 - Servicio de almacenamiento optimizado para consumo analítico.
- Diseñar e implementar servicios de visualización que permitan la exploración interactiva de los datos procesados.

3.3 Metodología de desarrollo

Las tareas del proyecto se desarrollarán como una serie de objetivos acompañados de un indicador clave de rendimiento (KPI) que permite verificar su cumplimiento, y de las acciones necesarias para alcanzarlo. Los valores numéricos de cada KPI se han fijado a partir de rangos razonables de la práctica de ingeniería y de la literatura revisada en el estado del arte; se validarán y se recalibrarán de ser necesario a partir de las pruebas realizadas.

3.3.1. Objetivo 1: Garantizar la persistencia de telemetría rápida y fiable

KPI: tiempo inferior a 25 segundos entre la publicación de evento y su persistencia final, para el percentil 95 de los mensajes MQTT validados, con una tasa de pérdida inferior al 0.1 %.

Acciones: instrumentar el simulador MQTT y desplegar Mosquitto como broker MQTT con un servicio de bridge MQTT-Kafka; implementar los servicios de Kafka, Spark Structured Streaming y el doble sumidero operacional / analítico; medir las latencias desde la publicación de los datos (este KPI incluye un estimado del tiempo de transmisión entre el dispositivo IoT y el broker Mosquitto, que en el caso de los dispositivos NBloT puede llegar a tardar hasta 20 segundos en sitios de mala cobertura).

3.3.2. Objetivo 2: Garantizar la gobernanza del esquema de datos

KPI: 100 % de eventos validados correctamente contra el esquema Avro registrado (sin eventos invalidados por error), y 0 % de fallos de deserialización.

Acciones: definir el esquema Avro inicial de telemetría; configurar las reglas de compatibilidad de esquema en Apicurio; diseñar y ejecutar una prueba verificando que no se interrumpe el flujo ni se pierden mensajes por error de validación o deserialización.

3.3.3. Objetivo 3: Procesar y enriquecer los datos en flujo con baja latencia

KPI: procesamiento de los micro-lotes de Spark Structured Streaming con una latencia estable menor a 2 segundos.

Acciones: implementar el proceso de Spark Structured Streaming (DataFrame API) con ventanas de agregación temporal y *watermarking* para el tratamiento de datos tardíos; medir tiempos de procesamiento de las consultas a Kafka.

3.3.4. Objetivo 4: Ofrecer visualización diferenciada operacional y analítica

KPI: visualización de datos con una latencia menor o igual a 5 segundos mediante dashboards de Grafana y páginas de Power BI que muestren consumo energético, eficiencia operativa, detección de anomalías y análisis histórico.

Acciones: implementar los dashboards operacionales de Grafana sobre los datos de TimescaleDB; implementar las páginas analíticas de Power BI sobre los datos en PostgreSQL; definir y calcular los datos requeridos por los tipos de análisis; validar la latencia de actualización de las visualizaciones.

3.3.5. Objetivo 5: Validar la resiliencia del sistema

KPI: los componentes del pipeline se recuperan automáticamente en menos de 60 segundos ante un fallo, sin pérdida de datos.

Acciones: simular la caída de un servicio (por ejemplo, el broker de mensajería o una de las bases de datos) y medir el tiempo de recuperación y la tasa de pérdida de datos.

3.4 Beneficios del proyecto

El desarrollo de este proyecto aporta un trabajo académico evaluable que documenta la implementación completa de una arquitectura Kappa para IoT basada en herramientas de código abierto, cerrando parcialmente los vacíos identificados en el estado del arte respecto a la gobernanza de esquemas y la separación de sumideros operacional y analítico.

Adicionalmente, el pipeline resultante queda disponible como sistema replicable sin dependencia de proveedores propietarios, lo que facilita su adopción como punto de partida en proyectos similares de monitorización energética.

Capítulo 4. ÉTICA Y COMPROMISO SOCIAL

Este capítulo recoge el compromiso del trabajo con los principios de igualdad de género y su contribución a los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de Naciones Unidas.

4.1 Igualdad de género

Este trabajo se alinea con los principios de igualdad de género y respeto a la diversidad. Por su naturaleza (el desarrollo de un pipeline de procesamiento de datos de telemetría energética a partir de un conjunto de datos público de mediciones de contadores, sin participación de sujetos humanos) no requiere consideraciones de muestreo poblacional ni plantea riesgos de sesgo respecto a colectivos protegidos. Su desarrollo y redacción se han realizado garantizando un entorno de trabajo respetuoso, con tolerancia cero hacia cualquier forma de discriminación, y empleando un lenguaje inclusivo y no sexista en toda la memoria.

El ámbito de la ingeniería de datos y de las infraestructuras de código abierto sigue siendo, en la actualidad, un entorno con una representación desigual de mujeres, tanto en su historia (con frecuencia invisibilizada, desde las pioneras de la programación hasta las primeras analistas de sistemas) como en su composición actual. Iniciativas como Women in Big Data o los programas de mentoría en comunidades de software libre buscan corregir ese desequilibrio estructural.

Este trabajo contribuye modestamente en esa dirección por la vía de la accesibilidad: al basarse en herramientas de código abierto y en un conjunto de datos público, y al documentar de forma completa y reproducible su metodología (arquitectura, decisiones de diseño y procedimientos de medición), elimina barreras económicas y de acceso privilegiado a la información que tradicionalmente han filtrado quién puede participar en el aprendizaje y la práctica de la ingeniería de datos en streaming. Un pipeline que cualquiera puede clonar, ejecutar, estudiar y extender sin depender de licencias comerciales ni de servicios gestionados de pago es, por construcción, más accesible que uno reservado a quienes ya cuentan con presupuesto o acceso privilegiado dentro de la industria.

4.2 Objetivos de Desarrollo Sostenible

Este trabajo se relaciona principalmente con los Objetivos de Desarrollo Sostenible 9 (Industria, Innovación e Infraestructura), 7 (Energía Asequible y No Contaminante) y 11 (Ciudades y Comunidades Sostenibles).

Respecto al ODS 9, el proyecto construye una infraestructura de procesamiento de datos en tiempo real íntegramente basada en tecnologías de código abierto (Mosquitto, Apache Kafka,

Apicurio, Apache Spark, TimescaleDB, PostgreSQL, Grafana, Docker), documentada y desplegable mediante contenedores, cuyo patrón arquitectónico (ingesta MQTT, gobernanza de esquemas y doble sumidero operacional/analítico) es directamente transferible a otros sectores industriales que requieran monitorización de dispositivos IoT, como el mantenimiento predictivo o la telemetría de flotas. La validación metodológicamente honesta de sus resultados (pruebas de carga y de recuperación ante fallos medidas y reproducibles, en lugar de cifras estimadas) promueve además una cultura de innovación rigurosa y transparente.

Respecto al ODS 7, el sistema desarrollado procesa telemetría de consumo eléctrico y de agua (fría y caliente) de edificios, calculando índices de eficiencia energética, perfiles de carga y detección de anomalías de consumo. Este tipo de instrumentación es la base técnica sobre la que se apoyan las estrategias de gestión activa de la demanda y de reducción del consumo fantasma, contribuyendo indirectamente a un uso más eficiente y sostenible de la energía en el parque de edificios.

Respecto al ODS 11, la monitorización granular del consumo energético de edificios (a nivel de emplazamiento, tipo de medidor y ventana temporal) constituye una capacidad habilitadora para la gestión sostenible de infraestructuras urbanas, permitiendo identificar patrones de consumo ineficiente y priorizar intervenciones de mejora energética en el parque construido.

De forma complementaria, el trabajo guarda relación con el ODS 4 (Educación de Calidad), en la medida en que el sistema completo (código, configuración de despliegue y documentación de las decisiones de diseño y de los resultados medidos) queda disponible como material formativo reutilizable para la enseñanza de la ingeniería de datos en streaming aplicada a un caso de uso real.

Capítulo 5. DESARROLLO DEL PROYECTO

5.1 Planificación del proyecto

El proyecto se desarrolló a lo largo de tres meses, entre junio y agosto de 2026, mediante fases consecutivas e incrementales. El esfuerzo total fue de 330 horas, a un ritmo aproximado de 28 horas semanales (112 horas mensuales). La Tabla 5.1 recoge el cronograma con las actividades realizadas y su esfuerzo.

Actividad	Periodo	Horas	Entregable
Estudio de los antecedentes y estado del arte	Jun. 2026	30	Revisión bibliográfica
Búsqueda y adaptación de la fuente de datos; establecimiento de tareas y objetivos	Jun. 2026	5	<code>prepare_ashrae.py</code> , objetivos y KPIs
Selección de herramientas y diseño de la arquitectura	Jun. 2026	5	Documento de diseño
Desarrollo técnico y pruebas iniciales de la sección de procesamiento	Jun.–Jul. 2026	80	Artefactos en <code>bridge/</code> , <code>common/</code> , <code>docker/</code> , <code>schemas/</code> , <code>simulator/</code> y <code>spark/</code>
Validación, pruebas y ajustes de la sección de procesamiento	Jul. 2026	100	Informe de resultados: pérdida de mensajes, latencia de persistencia p50/p95, soporte de carga, tiempo de recuperación ante fallo
Creación, pruebas y ajustes de la sección de visualización	Jul.–Ago. 2026	30	Artefactos en <code>grafana/</code> y <code>powerbi/</code>
Creación y prueba de scripts auxiliares	Ago. 2026	5	Artefactos en <code>tools/</code> y <code>setup.sh</code>
Redacción de la memoria	Jun.–Ago. 2026	75	Este documento
Total		330	

Tabla 5.1. Cronograma y esfuerzo de las actividades del proyecto

5.2 Descripción de la solución, metodologías y herramientas empleadas

5.2.1. Visión general de la arquitectura

El sistema implementa una arquitectura Kappa de flujo único con una bifurcación final hacia dos sumideros, tal y como muestra la Figura 5.1. Los componentes son servicios independientes, desacoplados mediante contenedores Docker, y la comunicación entre productores y consumidores de datos se ejecuta usando Kafka como log reproducible, mientras que la gobernanza de esquemas se resuelve a través del registro Apicurio.

El simulador (`mqtt_simulator.py`) publica telemetría hacia Eclipse Mosquitto, que actúa úni-

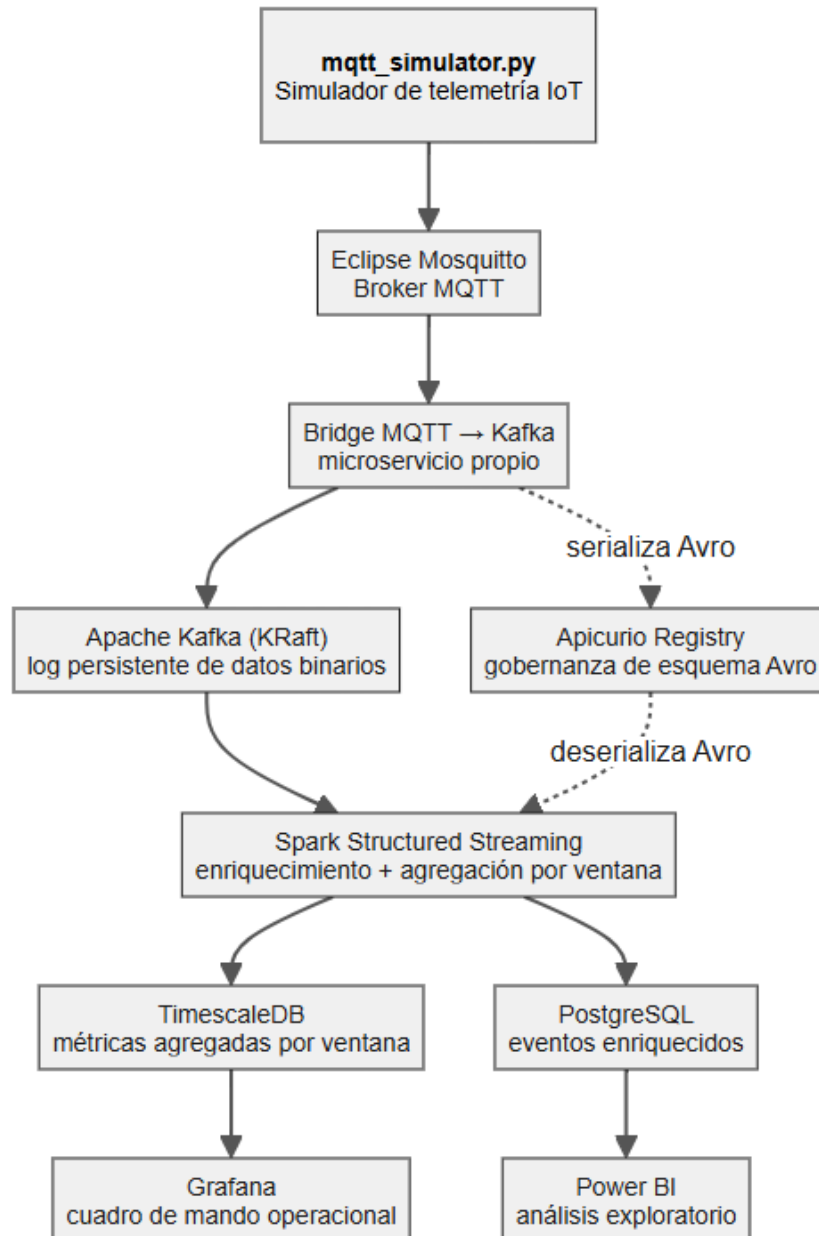


Figura 5.1. Arquitectura del sistema: flujo único Kappa (simulador, Mosquitto, bridge, Kafka, Spark) con bifurcación hacia el doble sumidero operacional y analítico. Elaboración propia.

camente como broker de ingesta MQTT. El bridge, un microservicio de diseño propio, se suscribe a los mensajes desde Mosquitto, los serializa contra Apicurio Registry conforme al esquema Avro vigente y los publica en Kafka.

La naturaleza de Kafka es de un log persistente de bytes crudos, ajeno a la estructura interna

de los mensajes almacenados y distribuidos. Es por ello que el bridge, en la escritura, y Spark Structured Streaming, en la lectura, son quienes dialogan directamente con Apicurio para serializar y deserializar las cadenas de bytes almacenadas y distribuidas por Kafka.

Spark Structured Streaming consume cada mensaje mediante dos consultas independientes sobre el mismo flujo de Kafka; en una de ellas enriquece los mensajes y agrega métricas por ventana temporal hacia TimescaleDB para el consumo operacional en Grafana, y en la otra consulta guarda los mensajes hacia PostgreSQL para el consumo analítico con Power BI. Al tratarse de consultas independientes, cada una mantiene su propio *checkpoint* y su propia tolerancia a fallos, de modo que la caída de una consulta no interrumpe a la otra.

5.2.2. Fuente de datos utilizada

Los datos para este proyecto provienen de la competición Kaggle ASHRAE – Great Energy Predictor III [18], un subconjunto del proyecto Building Data Genome 2 que registra el consumo energético por hora de 1.449 edificios reales en 16 emplazamientos desde el 2016-01-01 00:00h hasta el 2016-12-31 23:00h, con errores de medición (no hay datos sintéticos).

El script de preparación (`prepare_ashrae.py`) reduce el dataset original a un subconjunto de tres emplazamientos (identificados como 2, 3 y 5 con un total de 652 medidores y 5.682.185 lecturas de consumo), produciendo tres tablas Parquet:

- Una tabla de hechos con la lectura de cada medidor (Tabla 5.2).
- Una tabla de dimensión con los atributos de cada edificio (Tabla 5.3).
- Una tabla con el cálculo de los cuantiles según el histórico completo de medición de cada dispositivo (Tabla 5.4).

Variable	Uso en el sistema
<code>building_id</code>	Identificador del edificio; primer componente de la clave de partición en Kafka
<code>meter_type</code>	Tipo de medidor (electricidad, agua fría, agua caliente); segundo componente de la clave de partición en Kafka
<code>timestamp</code>	Instante en que se leyó el consumo; usado para el agregado por ventanas y el cálculo de ‘cierre de la ventana’ ante mensajes tardíos
<code>meter_reading</code>	Medición del consumo en kWh; usado para la detección de calidad, anomalías y métricas agregadas en los análisis

Tabla 5.2. Columnas de la tabla de hechos (`ashrae_telemetry.parquet`).

Variable	Uso en el sistema
building_id	Clave de unión con la tabla de hechos
site_id	Emplazamiento; primera clave de agrupación en las métricas agregadas
primary_use	Uso principal del edificio; segunda clave de agrupación de las métricas agregadas
square_feet	Superficie construida; base del cálculo de intensidad energética
year_built	Año de construcción; dejado para análisis futuros
floor_count	Número de plantas; dejado para análisis futuros

Tabla 5.3. Columnas de la tabla de dimensión (`ashrae_buildings.parquet`).

Variable	Uso en el sistema
building_id	Primera clave de unión con la tabla de hechos
meter_type	Segunda clave de unión con la tabla de hechos
baseline_p75	Percentil 75 histórico del medidor, usado para la detección de picos atípicos
baseline_iqr	Rango intercuartílico histórico del medidor

Tabla 5.4. Columnas de la línea base por sensor (`ashrae_sensor_baseline.parquet`).

5.2.3. Criterios de selección de herramientas

A continuación se documentan tres casos en los que se evaluaron herramientas usando los criterios planteados en el capítulo de Antecedentes y estado del arte, resultando en decisiones enmarcadas en los requisitos particulares de este proyecto.

Broker MQTT: Mosquitto con bridge propio frente a EMQX/HiveMQ Enterprise. Como recoge la Tabla 2.1, ningún broker MQTT con licencia aprobada por la Open Source Initiative ofrece un puente nativo hacia Kafka: esa función está disponible en las ediciones comerciales de EMQX [8] y HiveMQ [9]. Quedaban dos opciones compatibles con la filosofía de código abierto de este trabajo: adoptar un conector genérico como el conector MQTT de Stream Reactor para Kafka Connect [10], o desarrollar un microservicio de *bridging* propio. Se optó por la segunda opción porque permite asignarle al bridge responsabilidades adicionales más allá del simple reenvío de mensajes, sin depender de las limitaciones de configuración de un conector genérico. Adicionalmente, Eclipse Mosquitto se eligió sobre el resto de brokers de la Tabla 2.1 por ser la implementación de referencia de la Eclipse Foundation, con una huella de despliegue mínima adecuada para el volumen de mensajes de este proyecto.

Almacenamiento de series temporales: TimescaleDB frente a InfluxDB. La sección 2.2.1 advierte que los *benchmarks* de rendimiento entre los almacenamientos de series temporales proceden mayoritariamente de los propios fabricantes, por lo que la decisión en este proyecto no se apoyó en esas cifras sino en el ajuste con el resto de la arquitectura. Al ser TimescaleDB una extensión de PostgreSQL que ya se emplea en el sumidero analítico, ambos comparten cliente, dialecto SQL y patrón de escritura *upsert* en el mismo proceso de Spark, evitando

incorporar un lenguaje de consulta adicional (como Flux o InfluxQL) al proyecto.

Motor de procesamiento de flujos: Spark Structured Streaming frente a PyFlink. Aplicando los criterios de la sección 2.2.3 (relación entre la cadencia de los datos y los requisitos de latencia, madurez de la API en el lenguaje de implementación, unificación del modelo *batch/streaming*, y disponibilidad de talento) al caso concreto de este proyecto: el parque de medidores emite, verificado sobre el propio conjunto de datos, una lectura por hora y sensor, muy por debajo del umbral en el que las ventajas de latencia sub-segundo de un motor de flujo nativo como Flink resultan necesarias. A esto se suma que la API de PySpark tiene mayor madurez que la de PyFlink dentro del ecosistema Python, que Spark unifica el procesamiento por lotes y en flujo bajo el mismo modelo de *DataFrame*, y que su adopción en el ámbito profesional es más amplia.

5.2.4. Estructura del repositorio

```
tfm_boris_paternina/
|-- setup.sh                # Crea el entorno virtual, descarga el dataset Kaggle
|                           # y crea el subconjunto usando prepare_ashrae.py
|-- requirements.txt        # Dependencias Python del pipeline
|-- .sdkmanrc              # Versión de Java fijada para PySpark
|-- .gitignore
|-- pipeline/
|   |-- data/
|   |   |-- raw/            # Carpeta de descarga de ASHRAE Kaggle competition
|   |   |-- prepare_ashrae.py # Genera los archivos .parquet del pipeline
|   |-- docker/
|   |   |-- mosquitto/      # Configuración del broker MQTT
|   |   |-- grafana/        # Datasources y dashboards provisionados
|   |   |-- timescaledb/    # Esquema SQL de métricas y progreso de streaming
|   |   |-- postgres/       # Esquema SQL de eventos enriquecidos
|   |-- simulator/
|   |   |-- mqtt_simulator.py # Simulador del parque de medidores
|   |   |-- simulator_helper.py # Carga y reparto del dataset para el simulador
|   |-- bridge/
|   |   |-- mqtt_kafka_bridge.py # Microservicio bridge MQTT -> Kafka (Objetivo 1)
|   |   |-- Dockerfile      # Imagen del microservicio bridge
|   |-- schemas/
|   |   |-- telemetry_event_v1.avsc # Contrato de datos en Apicurio (Objetivo 2)
|   |   |-- register_schema.py      # Registro y gobernanza de esquemas en Apicurio
|   |-- spark/
|   |   |-- checkpoints/           # Estado de ejecución de las consultas a Kafka
|   |   |-- stream_processing.py    # Script de procesamiento PySpark (Objetivos 3 y 4)
|   |   |-- database_writers.py     # Escritura idempotente hacia los sumideros
|   |   |-- monitoring.py           # Supervisión del proceso Spark
|   |-- common/                    # Utilidades compartidas: logging, conexiones, Apicurio
|   |-- logs/                       # Logs de ejecución
|   |-- tools/                      # Scripts de medición (KPIs), prueba de fallos, demo.py
|   |-- .env.example                # Plantilla con credenciales de ejemplo para bases
|   |                               # de datos y configuración de tópicos Kafka
|   |-- docker-compose.yml          # Orquesta Mosquitto, Bridge, Apicurio, Kafka,
|   |                               # TimescaleDB, PostgreSQL y Grafana
```

```
|-- powerbi/  
|-- powerbi_dashboard_analitico.pbip          # Proyecto Power BI (formato PBIP)  
|-- powerbi_dashboard_analitico.Report/       # Definición de páginas y visuales  
|-- powerbi_dashboard_analitico.SemanticModel/ # Modelo semántico: tablas y relaciones
```

5.2.5. Preparación de los datos

Para la preparación de los datos usados en este proyecto, el script `prepare_ashrae.py` ejecuta, sobre los ficheros públicos de Kaggle, los siguientes pasos:

- **Filtrado por emplazamiento:** conserva únicamente los edificios cuyo `site_id` pertenece al subconjunto elegido (2, 3 y 5), descartando el resto mediante una unión provisional con la tabla de metadatos que actúa a la vez de filtro.
- **Decodificación del tipo de medidor:** traduce el código numérico original (0, 1 y 3) a su nombre (`electricity`, `chilledwater`, `hotwater`) usando la correspondencia declarada en la documentación de la competición.
- **Separación en tres tablas:** divide el resultado en la tabla de hechos, la tabla de dimensión y hace el cálculo de la tabla de cuartiles por sensor descritas en la sección 5.2.2 (Tablas 5.2, 5.3 y 5.4), cada una almacenada en un fichero Parquet independiente.
- **Verificación de la clave natural:** comprueba, en cada ejecución, que (`building_id`, `meter_type`, `timestamp`) identifica de forma única cada fila de la tabla de hechos; esta unicidad es la que garantiza la idempotencia del reprocesamiento de la arquitectura Kappa.
- **Reubicación temporal:** desplaza todos los timestamps del dataset original usando un único offset (`--fecha-final AAAA-MM-DD`) en el script Python, para que la última lectura caiga en la fecha indicada simplificando el análisis de los datos. Al ser un offset constante se conserva la cadencia y los ciclos diarios y estacionales del dataset original, y se exige una fecha presente o anterior para que ningún evento quede situado en el futuro.

5.2.6. Simulador de telemetría

El script `mqtt_simulator.py` ocupa el lugar de los 652 medidores reales representados en el subconjunto del dataset. Cada medidor publica una vez por hora, lo que supone 0,18 eventos por segundo considerando el total de medidores. Debido a lo anterior, el script usa el argumento `--acelerar` para incrementar el ritmo de publicación de los datos durante las pruebas de carga (en eventos por segundo), tal y como recoge la Tabla 5.5.

<code>--acelerar</code>	Cálculo eventos/segundo
2000	362
8000	1.448
40000	7.244

Tabla 5.5. Relación entre el factor de aceleración y el ritmo de publicación.

Una decisión de diseño relevante es publicar cada evento con un formato JSON en texto plano. El protocolo MQTT lo permite, ya que es agnóstico al formato del payload: no impone ninguna estructura ni codificación, y transporta indistintamente JSON, texto plano o binario.

```
{"building_id": "346", "meter_type": "electricity", "timestamp": "2026-02-13T17:00:00",  
"meter_reading": 40.99, "sim_publish_ts": 1788145201234}
```

La estructura de los campos de cada mensaje en formato JSON es la siguiente:

Campo	Significado
building_id	Identificador del edificio (texto)
meter_type	Tipo de medidor (electricity, chilledwater o hotwater)
timestamp	Timestamp al momento de tomar la lectura, sin zona horaria
meter_reading	Medición de consumo del medidor (decimal)
sim_publish_ts	Timestamp al momento de la publicación del dato, en formato epoch en milisegundos

Tabla 5.6. Campos del mensaje JSON publicado por el simulador.

En el terreno industrial/IIoT el formato más cercano al estándar es Sparkplug B, siendo entre un 50 y un 80 % más pequeño que el JSON equivalente al ser puramente binario. Por tanto se decide seguir la práctica recomendada: empezar con JSON en desarrollo, y migrar a un formato binario en producción cuando el ancho de banda lo exija.

5.2.7. Broker MQTT (Eclipse Mosquitto)

Mosquitto actúa únicamente como broker de ingesta MQTT: recibe la telemetría publicada por el simulador y la deja disponible para que el bridge la consuma, sin ejecutar ninguna lógica o transformación. Se despliega mediante la imagen oficial de Docker `eclipse-mosquitto:2.0.22`, con el puerto 1883 (MQTT, el que usa el simulador) y el 9001 (solo para inspección manual del tráfico durante el desarrollo del proyecto).

La configuración (`docker/mosquitto/mosquitto.conf`) fija cuatro aspectos relevantes:

Autenticación: `allow_anonymous true`. Válido en el entorno de desarrollo local del proyecto; en un despliegue real se sustituiría por `password_file` o autenticación TLS mutua.

Persistencia: `persistence true`, con almacenamiento y autoguardado cada 30 segundos. En MQTT, un mensaje publicado con QoS 1 ('al menos una vez') permanece pendiente en el broker hasta que el suscriptor lo confirma; la persistencia en disco es lo que permite que las sesiones y los mensajes QoS 1 todavía sin confirmar sobrevivan a un reinicio de Mosquitto, en vez de perderse junto con la memoria del proceso.

Límites de cola: `max_inflight_messages 200` fija cuántos mensajes QoS 1 puede retener el broker sin confirmar hacia un mismo suscriptor antes de detener el envío de más mensajes; esto es relevante para el bridge, que recibe la telemetría y debe confirmarla al mismo ritmo

que la reenvía hacia Kafka. Por su parte, `max_queued_messages 10000` fija el tamaño de la cola de salida que el broker mantiene para cada suscriptor; es donde se acumulan los mensajes que el broker no ha podido entregar porque el suscriptor está momentáneamente desconectado, o porque estando conectado confirma los mensajes más despacio de lo que le llegan mensajes nuevos. Superado el límite de 10.000 mensajes en cola, los siguientes que lleguen para ese suscriptor se descartan.

Tamaño del mensaje: `message_size_limit 0` fija el tamaño máximo del payload que el broker acepta desde el publicador. Con 0 (el valor por defecto) no se impone ningún límite más allá del tope que ya fija el protocolo MQTT. Dado que el payload JSON del simulador `mqtt_simulator.py` ronda los 145 bytes en este proyecto, este límite solo se deja por claridad de la configuración.

El archivo `docker-compose.yml` comprueba el estado del contenedor con `mosquitto_sub` en lugar de solo verificar que el puerto responde, confirmando que el broker acepta conexiones antes de considerarlo saludable.

5.2.8. Bridge MQTT → Kafka

El bridge (`mqtt_kafka_bridge.py`) se suscribe a los mensajes `iot/#` en Mosquitto, valida cada mensaje contra el esquema Avro vigente en Apicurio Registry, lo serializa a una cadena de bytes y lo publica en Kafka. Los mensajes que no superen la validación no se descartan: se desvían al tópico de mensajes inválidos de Kafka (`iot.telemetry.dlq`) junto con el motivo del rechazo, el tópico MQTT de origen (por ejemplo `iot/346/electricity/telemetry`), y un timestamp de cuando fue procesado por el bridge. DLQ significa *Dead Letter Queue*, un patrón estándar de sistemas de mensajería: en vez de descartar en silencio un mensaje que no se puede procesar, se desvía a una cola/tópico aparte con el motivo del rechazo, para que quede como evidencia y sea reprocesable posteriormente (G. Hohpe y B. Woolf [19]).

Un mensaje es enviado a `iot.telemetry.dlq` por dos vías distintas: si no encaja en el esquema Avro (por un campo ausente, tipo de dato incorrecto, o un tipo de medidor que no esté entre los declarados en el esquema), o si no supera una validación adicional implementada en el propio bridge (como una fecha futura que dañaría el watermark de Spark, o una lectura negativa o no finita). El mensaje que se guarde en el tópico de inválidos de Kafka estará en su formato original para su posterior análisis o reprocesamiento.

Del lado de Kafka, el bridge se configura con `acks='all'`, el nivel de confirmación más estricto disponible. Cada vez que el bridge publica un mensaje, Kafka le devuelve una confirmación de escritura antes de que el bridge lo dé por entregado; el valor de `acks` decide qué tiene que ocurrir en Kafka antes de que esa confirmación salga hacia el bridge.

- Con `acks=0`, Kafka no envía ninguna confirmación: el bridge da el mensaje por entregado en el momento de enviarlo, sin manera de saber si realmente llegó.
- Con `acks=1`, Kafka confirma en cuanto el líder de la partición escribe el mensaje en su propio registro, antes de que las réplicas lo copien.

- Con `acks='all'`, Kafka espera a que todas las réplicas en sincronía también tengan el mensaje, y solo entonces envía la confirmación al bridge.

De esta forma se garantiza que los mensajes publicados por el bridge no se dan por confirmados hasta que ya están almacenados en todas las réplicas, evitando una potencial pérdida de mensajes ante una caída de un broker de Kafka.

Ante el posible escenario de una caída del microservicio de bridge, la sesión persistente de Mosquitto con `QoS 1` y `max_queued_messages 10000` retiene los mensajes hasta la reconexión del microservicio para continuar con la validación, serialización y publicación en Kafka.

La clave con la que el bridge publica cada mensaje en Kafka es el propio sensor (`building_id: meter_type`), tal como se describió en la Tabla 5.2.

El bridge escala horizontalmente: el `command` declarado en `docker-compose.yml` incluye el argumento `"--shared-group=bridge-pool"`, que le pide al broker Mosquitto que reparta los mensajes entre las instancias del bridge en vez de duplicarlos para cada instancia por separado. De esta forma `docker compose up -d --scale bridge=N` levanta `N` réplicas que se reparten el flujo.

5.2.9. Registro de esquemas (Apicurio Registry)

Apicurio Registry gobierna el contrato de datos del proyecto. Se despliega mediante la imagen oficial de Docker `apicurio/apicurio-registry:3.3.1`, con una imagen aparte para la interfaz web (`apicurio/apicurio-registry-ui:3.3.1`).

Para que Apicurio pueda gobernar el esquema de validación de los datos, es necesario registrarlo antes del arranque del pipeline: se ejecuta manualmente la utilidad `register_schema.py`, que toma el contrato ya definido en `pipeline/schemas/telemetry_event_v1.avsc` y lo publica en Apicurio, estableciendo así, en un arranque desde cero, la primera versión del esquema en el registro.

`telemetry_event_v1.avsc` declara los cinco campos usados por el simulador de telemetría de la Tabla 5.6; por tanto la estructura de campos del esquema es la siguiente (se omitieron los valores de `'doc'` en este ejemplo):

```
{
  "type": "record",
  "name": "TelemetryEvent",
  "namespace": "es.uea.tfm.iot",
  "doc": "...",
  "fields": [
    {
      "name": "building_id",
      "type": "string",
      "doc": "..."
    },
    {
      "name": "meter_type",
      "type": {
```

```
    "type": "enum",
    "name": "MeterType",
    "symbols": ["electricity", "chilledwater", "steam", "hotwater", "unknown"],
    "default": "unknown",
    "doc": "...",
  },
  "doc": "...",
},
{
  "name": "timestamp",
  "type": {"type": "long", "logicalType": "timestamp-millis"},
  "doc": "...",
},
{
  "name": "meter_reading",
  "type": "double",
  "doc": "...",
},
{
  "name": "sim_publish_ts",
  "type": {"type": "long", "logicalType": "timestamp-millis"},
  "doc": "...",
}
]
}
```

El que se registre el esquema como un paso manual e independiente sigue una práctica recomendada en la industria para entornos de producción. Dejar que cada productor de mensajes registre automáticamente su propio esquema en Apicurio es cómodo en principio, pero delega el control de la evolución de esquemas en vez de ser un proceso supervisado. Por tanto, la práctica recomendada es publicar los esquemas mediante un mecanismo controlado y externo a la aplicación [20].

Los aspectos más importantes del funcionamiento de Apicurio dentro del pipeline son los siguientes:

- Apicurio comprueba que cada esquema que se le envía para su registro sea sintácticamente válido para serializar los mensajes a Avro. Adicionalmente comprueba que cada esquema nuevo sea compatible con todas las versiones de esquemas anteriores, no solo con el último. Si el esquema nuevo no pasa estas comprobaciones, se rechaza el registro.
- El bridge arranca y le pide el esquema más reciente a Apicurio: hace una llamada por HTTP a la API (`get_latest_version` sobre el subject `iot.telemetry.raw-value`) y recibe de vuelta dos cosas: el `schema_id` y el contenido del esquema. Esto ocurre una sola vez, al arrancar el proceso, antes de leer el primer mensaje de Mosquitto.
- Spark hace su propia llamada por HTTP a Apicurio, para obtener copia de todos los `schema_id` existentes. El bridge y Spark no se coordinan entre sí para estas llamadas; cada uno le pregunta a Apicurio por su cuenta.
- A partir de ahí, ninguno de los dos vuelve a llamar a Apicurio por cada mensaje. Tanto el bridge como Spark guardan los esquemas en memoria y los reutilizan durante toda

la ejecución.

- El bridge usa su copia del esquema más reciente dado por Apicurio para serializar cada mensaje válido en Avro, y le antepone la cabecera de cinco bytes (un byte mágico más el `schema_id`) antes de publicarlo en Kafka como cadena de bytes.
- Spark usa su copia de todos los esquemas para deserializar cada mensaje (cadena de bytes) que lee de Kafka, comparando el `schema_id` que trae la cabecera de cada mensaje contra el de los esquemas disponibles; si no encuentra el esquema, detiene el proceso en vez de deserializar con errores la cadena de bytes.
- Apicurio, por su parte, no guarda esquemas en memoria ni en una base de datos externa. Todos los esquemas registrados son almacenados en un tópico interno de Kafka (almacenamiento `kafkasql`), así que estos siguen disponibles aunque se reinicie el contenedor de Apicurio.

5.2.10. Apache Kafka

Kafka es el broker de mensajería sobre el que se apoya la arquitectura Kappa de flujo único descrita en la visión general (sección 5.2.1): su naturaleza es la de un log persistente de bytes crudos, ajeno a la estructura interna de los datos almacenados y distribuidos. Se despliega en modo KRaft con la imagen oficial de Docker `apache/kafka:4.3.1` como un solo nodo para este proyecto, con lo cual el mismo nodo combina los roles de *broker* y *controller*. Al tratarse de un solo nodo, el factor de replicación es necesariamente uno, por tanto no hay réplicas de las particiones a las que Kafka pueda recurrir si el nodo cae. Es la topología adecuada para el desarrollo y las pruebas de este proyecto, pero un despliegue real exigiría más de un broker para tener tolerancia real a fallos.

Cada mensaje recibido por Kafka desde el bridge es almacenado en su propia partición mediante la clave de partición compuesta de `building_id:meter_type`, que identifica de forma única a cada medidor, garantizando que todas sus lecturas se almacenen en la misma partición y se procesen en el orden en que se publicaron, tal como lo necesitan las ventanas de agregación de Spark.

Se utilizan dos tópicos para el enrutamiento de los mensajes: el tópico `iot.telemetry.raw` contiene los mensajes válidos publicados por el bridge, y el tópico `iot.telemetry.dlq` contiene los mensajes rechazados, descritos en la sección 5.2.8. Estos tópicos se crean de forma automática; tanto sus nombres como el número de particiones y el factor de replicación se configuran con los valores establecidos en el archivo `pipeline/.env`.

Tanto el tópico `iot.telemetry.raw` como `iot.telemetry.dlq` retienen los mensajes durante siete días. Esa ventana es la que sostiene la recuperación sin pérdida de datos: un consumidor de datos de Kafka que se detiene puede reiniciarse sin perder eventos dentro de ese margen de tiempo.

En la configuración de despliegue para Kafka en `docker-compose.yml` se establecen dos conexiones distintas con los otros servicios del sistema: `INTERNAL` (`kafka:9092`), que es usada por la imagen del bridge, y `EXTERNAL` (`localhost:29092`), que es usada por Spark.

Esto es necesario porque Spark no se ejecuta en un contenedor, como se explicará en la sección 5.2.11.

5.2.11. Apache Spark Structured Streaming

Spark Structured Streaming (`stream_processing.py`) es el único consumidor de Kafka: dos consultas de flujo independientes al tópic `iot.telemetry.raw` deserializan cada mensaje Avro usando el esquema correspondiente al `schema_id` del mensaje, los enriquecen cruzándolos con las tablas de referencia del proyecto, y los guardan en sus respectivos sumideros. Como cada consulta de flujo se realiza con su propio *checkpoint* que registra hasta qué punto se ha consumido el tópic, la caída de una consulta causada por una falla de PostgreSQL o TimescaleDB no interrumpe a la otra.

Las dos consultas comparten la misma lógica de enriquecimiento inicial de los datos: un join por difusión de Spark (*broadcast join*) con `ashrae_buildings.parquet` y con `ashrae_sensor_baseline.parquet`, ya que ambas tablas son lo bastante pequeñas para replicarse en la memoria de cada nodo sin necesidad de barajar (*shuffle*) los datos. Adicionalmente, la consulta hacia TimescaleDB agrega por ventanas temporales de una hora sobre el tiempo de evento (`timestamp`), agrupando siempre por tipo de medidor además de por emplazamiento y uso del edificio. Esta agregación por ventana de TimescaleDB exige una marca de agua (*watermark*) que le indique a Spark cuánto puede tardar un evento en llegar antes de dar una ventana por cerrada. La consulta hacia PostgreSQL almacena cada evento enriquecido de forma directa, por lo que no necesita ventanas de agregación con *watermark*.

La escritura en ambos sumideros se hace a través del script `database_writers.py`: este ejecuta un UPSERT (`INSERT ... ON CONFLICT DO UPDATE`) sobre la clave natural de cada tabla, en vez de un simple `INSERT`. De esta forma se hace idempotente el reprocesamiento del log de Kafka, condición necesaria en una arquitectura Kappa. Ante una caída breve de PostgreSQL o TimescaleDB, `database_writers.py` reintenta un número configurable de veces antes de dar la conexión por perdida.

Spark se ejecuta desde el entorno virtual del host y no como un servicio más del `docker-compose.yml` a diferencia del resto de componentes del pipeline. Una razón es que cada cambio en el código habría exigido reconstruir o volver a copiar la imagen del contenedor, mientras que ejecutándolo en local la depuración desde el editor es directa. Por otra parte, contenerizar el proceso de Spark exige requisitos de procesador y memoria que ya están comprometidos para los otros componentes del pipeline. Por tanto la contenerización queda como una tarea pendiente para una eventual puesta en producción.

5.2.12. TimescaleDB

TimescaleDB se despliega con la imagen `timescale/timescaledb:2.29.1-pg17-oss`. Contiene dos tablas, `telemetry_metrics` y `streaming_progress`, cuyas credenciales de acceso (usuario y contraseña) se leen de `pipeline/.env`, el mismo archivo usado por Kafka y PostgreSQL para sus configuraciones de funcionamiento.

Cada fila de `telemetry_metrics` corresponde a una ventana de una hora para un emplazamiento, un uso de edificio y un tipo de medidor; sus columnas son las siguientes:

Columna	Descripción
<code>window_start, window_end</code>	Inicio y fin de la ventana de una hora sobre el tiempo de evento (timestamp)
<code>site_id</code>	Emplazamiento del edificio; clave de agrupación
<code>primary_use</code>	Uso principal del edificio; clave de agrupación
<code>meter_type</code>	Tipo de medidor; clave de agrupación
<code>event_count</code>	Número de lecturas que caen dentro de la ventana
<code>distinct_buildings</code>	Número de edificios distintos que reportaron en la ventana, contados de forma aproximada porque Spark no admite el conteo exacto en agregaciones de flujo
<code>avg_reading</code>	Media de las lecturas de consumo de la ventana
<code>max_reading</code>	Máximo de las lecturas de consumo de la ventana
<code>sum_reading</code>	Suma de las lecturas de consumo de la ventana
<code>sum_square_foot</code>	Suma de la superficie construida de los edificios de la ventana
<code>avg_energy_intensity</code>	Se calcula dividiendo <code>sum_reading</code> entre <code>sum_square_foot</code>
<code>zero_count</code>	Cuántas lecturas de la ventana fueron cero
<code>anomaly_count</code>	Cuántas lecturas de la ventana se marcaron como anómalas
<code>ingested_at</code>	Instante en que la fila se escribió o actualizó en la base de datos, asignada por la propia base de datos

Tabla 5.7. Columnas de `telemetry_metrics`.

Además de `telemetry_metrics`, TimescaleDB también contiene `streaming_progress`, la tabla que registra el progreso de cada micro-lote de las dos consultas de Spark y que sostiene el KPI de latencia. A diferencia de `telemetry_metrics`, sus columnas no son agregaciones calculadas por el pipeline: son valores que la propia API de Spark Structured Streaming ya expone por cada micro-lote, y que el proceso se limita a extraer y guardar. Cada fila corresponde a un micro-lote de una de las dos consultas; sus columnas son las siguientes:

Columna	Descripción
trigger_ts	Instante en que Spark emitió el informe del micro-lote
run_id	Identificador de la ejecución del proceso
query_name	Nombre de la consulta que generó el informe (metricas-timescaledb o eventos-postgresql)
batch_id	Número del micro-lote dentro de esa consulta
num_input_rows	Filas leídas de Kafka en este micro-lote
input_rows_per_second	Ritmo de entrada de filas
processed_rows_per_second	Ritmo al que la consulta procesó las filas
duration_ms	Duración total del micro-lote
add_batch_ms	Parte de duration_ms dedicada a la escritura del micro-lote
query_planning_ms	Parte de duration_ms dedicada a planificar la consulta
sink_num_output_rows	Filas escritas en el sumidero
rows_dropped_by_watermark	Eventos descartados por llegar tarde respecto al <i>watermark</i>
offsets_behind	Mensajes en Kafka que la consulta no ha leído
event_time_max	Marca de tiempo de evento más reciente del micro-lote
watermark	Valor del <i>watermark</i> del micro-lote

Tabla 5.8. Columnas de streaming_progress.

5.2.13. PostgreSQL

PostgreSQL se despliega con la imagen postgres:17-alpine, en el puerto 5433 del host en vez del 5432 habitual, porque ese ya lo ocupa TimescaleDB (que también es un PostgreSQL). Sus credenciales, igual que las de TimescaleDB, se leen de pipeline/.env.

Contiene tres tablas:

- buildings: la tabla de dimensión de edificios descrita en la sección 5.2.2 (Tabla 5.3), cargada desde el fichero Parquet correspondiente.
- sensor_baseline: la tabla de cuartiles por sensor, también descrita en la sección 5.2.2 (Tabla 5.4).
- telemetry_events: la tabla de eventos individuales

Las columnas de telemetry_events no son agregaciones; conforman prácticamente el mismo esquema Avro usado por el bridge y por Spark, sin ningún campo calculado:

Columna	Descripción
building_id	Identificador del edificio; primer componente de la clave del evento
meter_type	Tipo de medidor; segundo componente de la clave del evento
event_time	Instante de la lectura; tercer componente de la clave del evento
meter_reading	La medición de consumo
sim_publish_ts	Instante de publicación del evento por el simulador <code>mqtt_simulator.py</code>
ingested_at	Instante en que la fila se escribió o actualizó en la base de datos, asignado por la propia base de datos

Tabla 5.9. Columnas de `telemetry_events`.

5.2.14. Grafana

Grafana se despliega con la imagen oficial de Docker `grafana/grafana:13.0.6`, en el puerto 3000, con dependencia de arranque sobre TimescaleDB. Se habilita el acceso anónimo de solo lectura (*Viewer*), pensado para la visualización directa en la pantalla del navegador sin necesidad de introducir credenciales.

Grafana se conecta a **TimescaleDB**, cuya tabla `telemetry_metrics` es la fuente de datos usada en todos los paneles de los dashboards, teniendo como única excepción los paneles que registran la latencia extremo a extremo, los cuales usan datos de **PostgreSQL**.

En `http://localhost:3000/` se presentan tres dashboards, agrupados bajo la carpeta TFM:

1. Estado del pipeline. Diseñado para vigilar la salud del sistema en tiempo real, con actualización por defecto de 5 segundos.

Panel	Descripción
Latencia extremo a extremo (actual y evolución en el tiempo)	Percentiles de <code>ingested_at - sim_publish_ts</code> de la tabla <code>telemetry_events</code>
Antigüedad del último dato	Segundos desde el <code>ingested_at</code> de la ventana más reciente
Eventos procesados	Suma de la columna <code>event_count</code> en el rango visible del dashboard
Ventanas escritas	Conteo de filas en el rango visible del dashboard
Ritmo de escritura	Suma de <code>event_count</code> y conteo de filas, agregados por intervalo de tiempo

Tabla 5.10. Paneles del dashboard 'Estado del pipeline'.

2. Consumo energético. Con actualización por defecto de 30 segundos y un selector de tipo de medidor para alternar entre electricidad, agua fría y agua caliente.

Panel	Descripción
Demanda pico	Máximo del consumo horario agregado (<code>sum_reading</code>) en el rango visible del dashboard
Consumo base	Mínimo del consumo horario agregado (<code>sum_reading</code>) en el rango visible del dashboard
Factor de carga	Cociente entre la media y el máximo del consumo horario agregado (<code>sum_reading</code>) en el rango visible del dashboard
Edificios reportando	Valor máximo de <code>distinct_buildings</code> en el rango visible del dashboard
Consumo por tipo de medidor	Suma de <code>sum_reading</code> , agregada por intervalo de tiempo y agrupada por <code>meter_type</code>
Perfil de carga por hora del día	Suma de <code>sum_reading</code> por hora del día (0 a 23) para todas las fechas del rango visible
Consumo por emplazamiento	Suma de <code>sum_reading</code> , agregada por intervalo de tiempo y agrupada por <code>site_id</code>

Tabla 5.11. Paneles del dashboard 'Consumo energético'.

3. Calidad y anomalías. Centrado en la salud de los medidores, con actualización por defecto de 30 segundos.

Panel	Descripción
Medidores con lecturas cero	Cociente entre la suma de <code>zero_count</code> y la suma de <code>event_count</code> en el rango visible del dashboard
Anomalías detectadas	Cociente entre la suma de <code>anomaly_count</code> y la suma de <code>event_count</code> en el rango visible del dashboard
Lecturas válidas	Cociente entre las lecturas que no son cero ni anómalas y el total de <code>event_count</code> en el rango visible del dashboard
Ceros y anomalías en el tiempo	Suma de <code>zero_count</code> y de <code>anomaly_count</code> , agregada por intervalo de tiempo
Grupos con más lecturas a cero	Ranking de emplazamiento, uso de edificio y tipo de medidor con más <code>zero_count</code> acumulado en el rango visible del dashboard
Ventanas con anomalías recientes	Detalle de las últimas ventanas con <code>anomaly_count</code> mayor que cero, junto con la lectura máxima de cada una

Tabla 5.12. Paneles del dashboard 'Calidad y anomalías'.

5.2.15. Power BI

El informe de Power BI está almacenado en formato PBIP (*Power BI Project*) dentro de la carpeta `powerbi/` del proyecto. Su modelo semántico se conecta en modo *DirectQuery* a PostgreSQL, consultando los datos más recientes tanto al inicio como al interactuar con las diversas opciones y filtros; cada página tiene además la opción de activar la actualización automática.

El informe contiene tres páginas:

1. Consumo de Energía.

- Consumo en kWh por Tiempo de medición y Tipo de medidor (gráfico de líneas)
- Consumo en kWh por Emplazamiento (gráfico de barras)
- Total de edificios analizados (tarjeta)

Filtros interactivos: Tipo de medidor, Uso de edificación y Rango de fechas inicio-fin de medición.

2. Eficiencia Eléctrica. La página filtra únicamente medidores de electricidad:

- Electricidad total consumida en kWh (tarjeta)
- Total de edificios analizados (tarjeta)
- Eficiencia de consumo en kWh/m² por Uso de edificación (gráfico de barras)
- Top 10 edificios menos eficientes con sus valores en kWh/m² (gráfico de barras)
- Top 10 edificios más eficientes con sus valores en kWh/m² (gráfico de barras)

Filtro interactivo: Rango de fechas inicio-fin de medición.

3. Detección de Anomalías. La página filtra únicamente mediciones anómalas:

- Total Edificios analizados (tarjeta)
- Total Anomalías registradas (tarjeta)
- Tabla de detalle con edificio, tipo de medidor, timestamp de medición, lectura de consumo y límite superior de consumo
- Total de Anomalías por Fecha (gráfico de columnas)

Filtro interactivo: Rango de fechas inicio-fin de medición.

5.3 Recursos requeridos

Los recursos utilizados para la ejecución del proyecto fueron:

- Ordenador personal (Windows 11, procesador Intel Core i5 a 1,30 GHz, 16 GB de RAM), con WSL2 (Ubuntu 24.04 LTS) para el entorno de desarrollo.
- Python con entorno virtual dedicado (`venv + pip`) y Java 21 LTS (Temurin), gestionado con SDKMAN.
- Librerías de código abierto: PySpark 4.2.0, pandas 2.3.3, PyArrow 22.0.0, paho-mqtt 2.1.0, aiomqtt 2.5.1, psycpg2-binary 2.9.9, fastavro 1.9.7, confluent-kafka[avro] 2.15.0, requests 2.32.3 y kafka-python 2.0.2.
- Docker con las imágenes oficiales de Eclipse Mosquitto 2.0.22, Apache Kafka 4.3.1, Apicurio Registry 3.3.1, TimescaleDB 2.29.1, PostgreSQL 17 (Alpine) y Grafana 13.0.6.
- Conjunto de datos público ASHRAE – Great Energy Predictor III (Kaggle) [18].
- Power BI Desktop.
- Editor de código (VS Code), Git/GitHub y LaTeX (plantilla UEM) para la elaboración de esta memoria.

5.4 Presupuesto

La Tabla 5.13 es un estimado del coste económico. El software es de código abierto o de uso gratuito para este proyecto, por lo que el mayor coste es el tiempo de trabajo.

Tipo de coste	Valor	Comentarios
Horas de trabajo en el proyecto	330 h × 25 €/h = 8.250 €	Estimación de mercado de un analista de datos junior; desglose de horas en la Tabla 5.1.
Equipo técnico utilizado	750 €	Ordenador portátil personal (valor de mercado si hubiera que adquirirlo nuevo).
Software utilizado	0 €	VS Code, Git/GitHub, Power BI Desktop y las librerías (Python, PySpark, Mosquitto, Kafka, Apicurio, TimescaleDB, PostgreSQL, Grafana y Docker) son de uso libre.
Datos	0 €	Conjunto de datos público ASHRAE, vía Kaggle.
Estudios e informes	0 €	Literatura académica de libre acceso o a través de la universidad.
Total	9.000 €	

Tabla 5.13. Costes del proyecto.

5.5 Viabilidad

5.5.1. Viabilidad técnica

El proyecto se ha desarrollado íntegramente con herramientas de código abierto ampliamente adoptadas en la industria (Kafka, Spark, PostgreSQL, Docker), sin dependencia de proveedores propietarios ni de infraestructura en la nube. El aprovisionamiento declarativo de cada componente permite reconstruir el sistema completo desde el repositorio en cualquier máquina con Docker y Java instalados, sin pasos de configuración manual. La arquitectura Kappa de flujo único, con los componentes desacoplados en microservicios independientes, facilita sustituir o escalar cualquier pieza concreta sin rediseñar el resto del sistema.

5.5.2. Viabilidad económica

El coste económico del proyecto ha sido mínimo, ya que no ha sido necesario adquirir software, licencias ni infraestructura en la nube: la Tabla 5.13 sitúa el coste total en 9.000 €, dominado casi en su totalidad por el tiempo de trabajo (330 horas). Esta inversión se justifica por el valor académico y formativo del proyecto, más allá de un producto con fin comercial.

5.5.3. Sostenibilidad

El diseño modular del sistema permite incorporar nuevos sumideros, consultas de enriquecimiento o dashboards sin modificar los componentes ya existentes. El uso de un conjunto de datos público (ASHRAE) y de herramientas open-source sin coste de licencia facilita también la reproducibilidad del proyecto y su adopción como punto de partida en trabajos de monitorización energética similares.

5.6 Resultados del proyecto

Los resultados conseguidos en el desarrollo del proyecto confirman el cumplimiento de los objetivos planteados en el Capítulo 3.3. Debe aclararse que todas las mediciones se realizaron con instancias únicas para cada servicio, haciendo que la interrupción y recuperación sin pérdida de datos de la sección 5.6.5 sucedan bajo una condición más adversa.

El simulador MQTT se ejecutó en modo acelerado, ya que el ritmo real del parque de medidores (652 medidores / 3600 segundos = 0,18 eventos por segundo) habría exigido semanas de ejecución continua para obtener comprobación de la correcta ingesta, validación y persistencia de los datos procesados.

Por ello, `mqtt_simulator.py` con la opción `--acelerar 2000`, se encargó de que cada medidor publicara cada 1,8 segundos en vez de cada hora (3600 segundos / 2000 = 1.8), y la velocidad del parque completo ascendió a 362 eventos por segundo (Tabla 5.5). Este script se encargó de que los 652 medidores no publicaran los eventos a la misma hora en punto como está originalmente en el dataset, haciendo que el parque entregara un flujo sostenido en lugar de ráfagas de 652 eventos.

5.6.1. Objetivo 1: Garantizar la persistencia de telemetría rápida y fiable

Indicador	Resultado	Objetivo
Eventos publicados / persistidos	50.000 / 50.000	—
Latencia de persistencia (mediana)	20,715 s	—
Latencia de persistencia (p95)	21,175 s	<25 s
Latencia de persistencia (máxima)	22,260 s	—
Tasa de pérdida	0,0 %	<0,1 %

Tabla 5.14. Resultado del Objetivo 1: persistencia de telemetría rápida y fiable.

Los valores de la Tabla 5.14 se obtuvieron sobre `telemetry_events` en PostgreSQL y los `offsets` del tópico `iot.telemetry.raw` de Kafka, tras publicar 50.000 eventos.

Los eventos **publicados** por el simulador se calcularon como la suma de todos los (`offset` final - `offset` inicial) de todas las particiones del tópico de Kafka (asumiendo la no pérdida de eventos entre el simulador, el broker y el bridge). Los eventos **persistidos** son el recuento

de filas de la tabla `telemetry_events`. La **tasa de pérdida** es el porcentaje de eventos publicados que no llegó a persistirse, es decir $(\text{publicados} - \text{persistidos}) / \text{publicados}$.

La **latencia de persistencia** de cada evento es el tiempo transcurrido desde `sim_publish_ts` (publicación) hasta `ingested_at` (persistencia), en segundos (Tabla 5.9). Sobre esa diferencia se calcularon la **mediana**, el **percentil 95** y el valor **máximo** para las 50.000 filas.

5.6.2. Objetivo 2: Garantizar la gobernanza del esquema de datos

Indicador	Resultado	Objetivo
Eventos validados contra <code>es.uea.tfm.iot.TelemetryEvent</code>	100 %	100 %
Eventos con error de validación (DLQ)	0	0

Tabla 5.15. Resultado del Objetivo 2: gobernanza del esquema de datos.

Los valores de la Tabla 5.15 se obtuvieron de Apicurio Registry y de los dos tópicos de Kafka implicados en la validación.

`es.uea.tfm.iot.TelemetryEvent` es el nombre completo del esquema Avro contra el que se valida toda la telemetría del pipeline. Los **eventos validados** son los mensajes presentes en el tópico `iot.telemetry.raw`, contados por el mismo método de *offsets* descrito para los eventos publicados del Objetivo 1, y los **eventos con error de validación** son los eventos del tópico de descartes `iot.telemetry.dlq` contados también de la misma forma. Todo mensaje que llega a `iot.telemetry.raw` lo hizo porque el bridge consiguió serializarlo contra el esquema registrado sin error; el que no encaja en el esquema o que no supera las validaciones del propio bridge, se desvía a `iot.telemetry.dlq` con su motivo de rechazo.

Se contaron 50.000 mensajes en `iot.telemetry.raw` y ninguno en `iot.telemetry.dlq`, de ahí el 100 %.

La ausencia de fallos de deserialización del lado de Spark se sostiene sobre el resultado del Objetivo 1: las 50.000 filas persistidas implican que Spark deserializó los 50.000 mensajes contra el esquema correspondiente a su `schema_id` sin una sola excepción, ya que un `schema_id` no reconocido habría detenido el proceso.

5.6.3. Objetivo 3: Procesar y enriquecer los datos en flujo con baja latencia

Flujo	Lotes	Mediana	p95	Máxima	Objetivo
Eventos individuales (sumidero analítico)	142	262 ms	385 ms	1.428 ms	<2 s
Métricas agregadas (sumidero operacional)	142	525 ms	747 ms	1.887 ms	<2 s

Tabla 5.16. Resultado del Objetivo 3: latencia de micro-lotes de Spark.

Los valores de la Tabla 5.16 proceden de `streaming_progress` de TimescaleDB (Tabla 5.8): la tabla que registra el progreso del procesamiento por flujo, con valores que la propia API de Spark Structured Streaming ya expone por cada micro-lote procesado.

Cada **flujo** de la tabla es una consulta independiente de Spark sobre Kafka: `eventos-postgresql` escribe cada evento individual y `metricas-timescaledb` escribe las métricas agregadas por ventanas. **Lotes** es el número de micro-lotes procesados en cada consulta; **Mediana** y **p95** son los percentiles 50 y 95 de `duration_ms`, y **Máxima** es el máximo tiempo de procesamiento entre todos los micro-lotes.

5.6.4. Objetivo 4: Ofrecer visualización diferenciada operacional y analítica

Dashboard	Paneles	Ventana	Actualización	Objetivo
1. Estado del pipeline	6	15 min	5 s	≤ 5 s
2. Consumo energético	7	7 días	30 s	—
3. Calidad y anomalías	6	7 días	30 s	—

Tabla 5.17. Resultado del Objetivo 4: configuración de los dashboards de Grafana.

Los valores de la Tabla 5.17 son la configuración de los tres *dashboards* de Grafana (sección 5.2.14): el número de **paneles**, la **ventana** de tiempo que muestra cada uno y su **intervalo de actualización**, que es cada cuánto Grafana vuelve a consultar los datos y repinta el *dashboard*. El *dashboard* operativo, “Estado del pipeline”, se actualiza cada 5 segundos, de modo que el dato mostrado nunca tiene más de 5 segundos de antigüedad; los otros dos presentan agregados de negocio sobre ventanas de siete días con actualización cada 30 segundos. La consulta de cada panel se resuelve en fracciones de segundo, y tanto la ventana como el intervalo de actualización son ampliamente configurables.

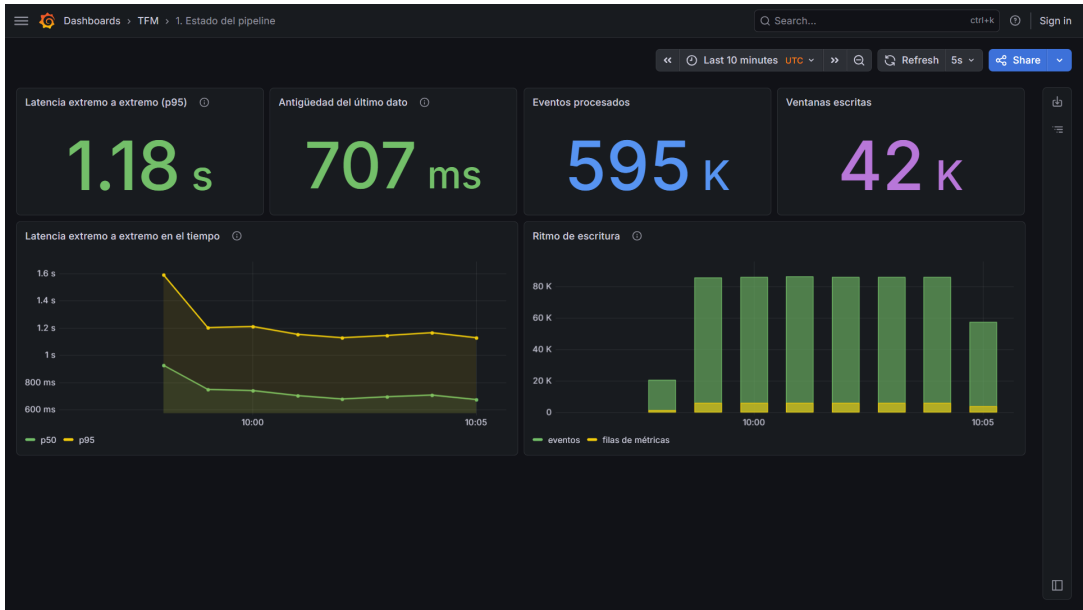


Figura 5.2. Dashboard 'Estado del pipeline'.

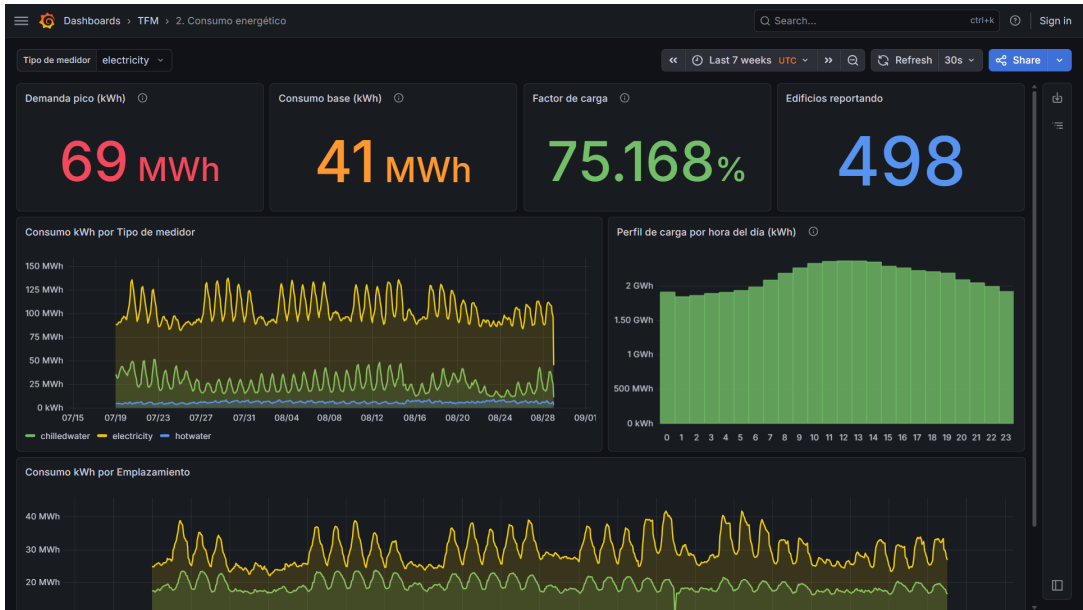


Figura 5.3. Dashboard 'Consumo energético'.

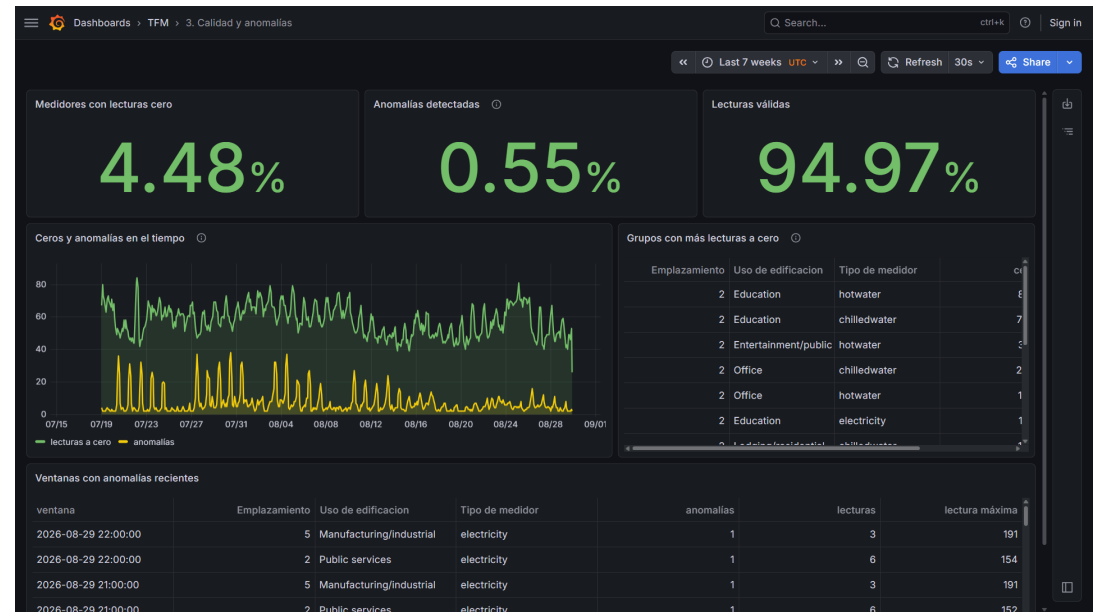


Figura 5.4. Dashboard 'Calidad y anomalías'.

En cuanto a Power BI (sección 5.2.15), el informe cubre las áreas exigidas por el KPI: consumo energético, eficiencia operativa, detección de anomalías y análisis histórico.

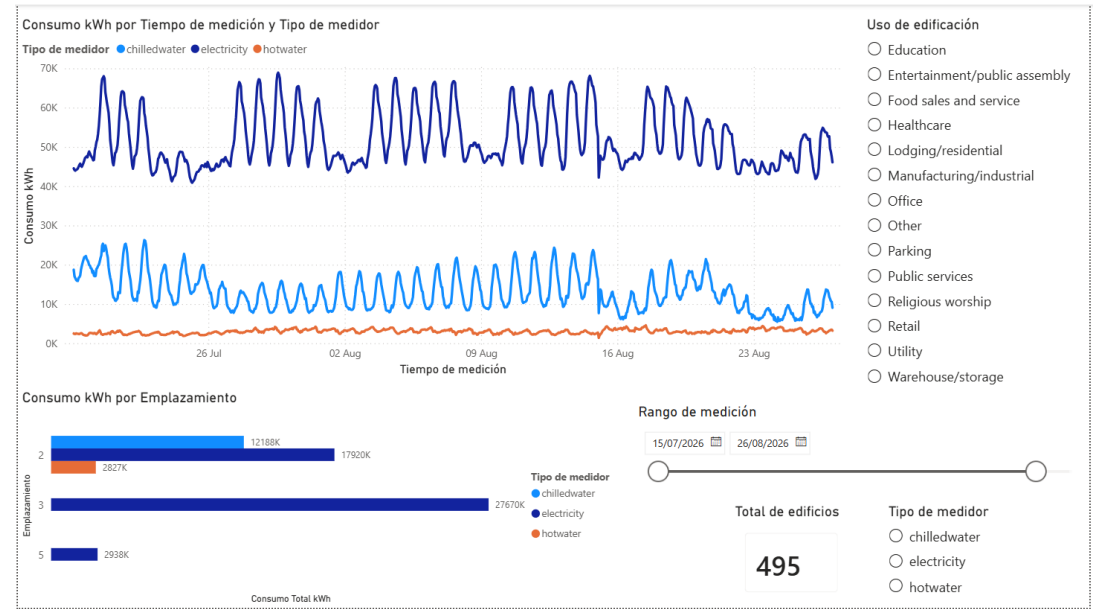


Figura 5.5. Página 'Consumo de Energía'.

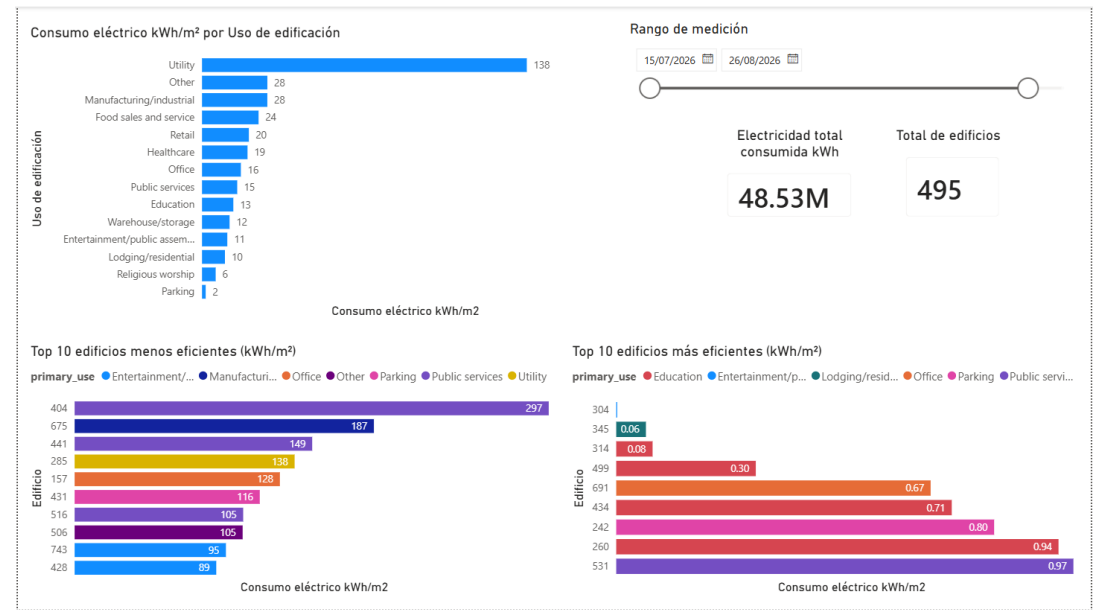


Figura 5.6. Página 'Eficiencia Eléctrica'.

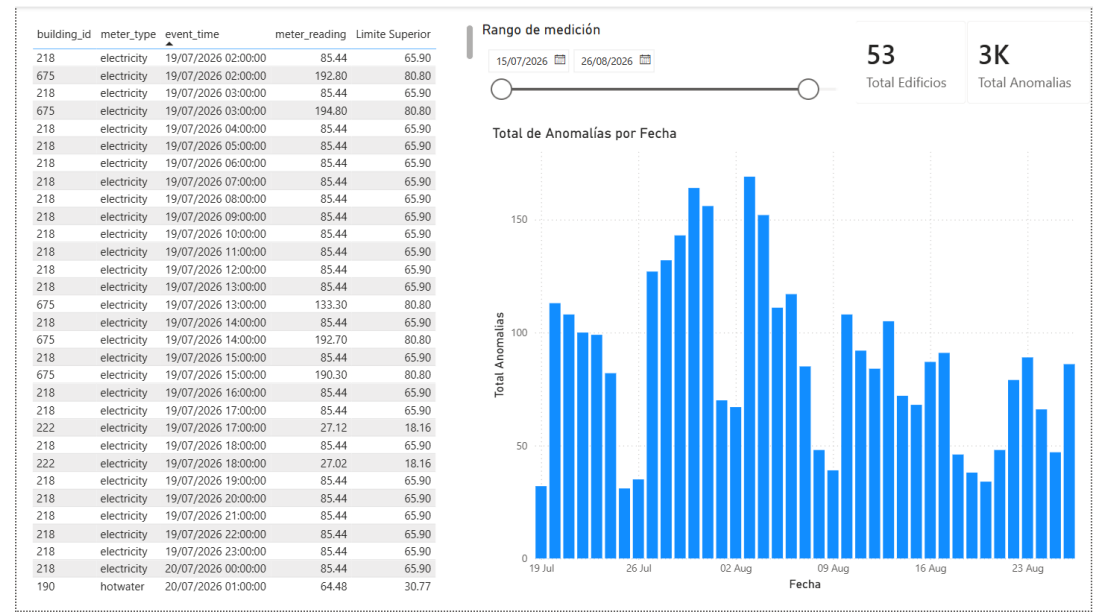


Figura 5.7. Página 'Detección de Anomalías'.

5.6.5. Objetivo 5: Validar la resiliencia del sistema

Los valores de la Tabla 5.18 se obtuvieron con el comando `python failover_test.py --target <servicio> --fallo oom` dentro de la carpeta `pipeline/tools/`.

Con el pipeline en marcha, el script simula 181 eventos por segundo, espera a ver flujo llegando al sumidero y entonces provoca el fallo del servicio bajando el límite de memoria del contenedor a 6 MB (por debajo de la memoria requerida), de modo que el núcleo mata su

proceso principal por falta de memoria. A continuación `failover_test` devuelve el límite a 2 GB, y la política `restart: unless-stopped` declarada en el `docker-compose.yml` vuelve a levantar el contenedor sin intervención. Desde ese instante se cronometra cuánto tarda en aparecer una fila nueva en la tabla afectada por ese fallo concreto: `telemetry_metrics` (TimescaleDB) cuando el servicio caído es TimescaleDB, y `telemetry_events` (PostgreSQL) en los demás servicios.

El fallo por OOM no puede forzarse sobre Mosquitto porque consume solo 3 MB. Para el servicio de Kafka, el fallo por OOM es un evento extremo que deja a su motor JVM en bucle de reinicios. Por tanto en ambos casos se usa `--fallo kill`.

Servicio interrumpido	--fallo	Recuperación	Tasa de pérdida	Objetivo
TimescaleDB	oom	3,3 s	0,0 %	<60 s
Mosquitto	kill	8,3 s	0,0 %	<60 s
Kafka	kill	14,4 s	0,0 %	<60 s
PostgreSQL	oom	2,4 s	0,0 %	<60 s
Bridge	oom	1,2 s	0,0 %	<60 s

Tabla 5.18. Resultado del Objetivo 5: pruebas de resiliencia del sistema.

La **tasa de pérdida** se calculó con el mismo método descrito para la tabla 5.14 del Objetivo 1: los `offsets` del tópico `iot.telemetry.raw` como eventos publicados, las filas de `telemetry_events` como eventos persistidos, y la tasa como $(\text{publicados} - \text{persistidos}) / \text{publicados}$.

Capítulo 6. EJECUCIÓN Y VALIDACIÓN DEL SISTEMA

Este capítulo describe cómo poner en marcha el pipeline y cómo reproducir las mediciones presentadas en el capítulo 5.6. Se ofrecen dos vías de arranque: una automática y una manual paso a paso, y se documentan las herramientas con las que se obtuvieron los resultados de los objetivos.

6.1 Requisitos previos

El sistema se ejecuta sobre Linux, WSL2 o macOS y las dependencias en detalle están en `requirements.txt` en la raíz del repositorio:

- Docker y Docker Compose para el stack de servicios.
- Java 17 o superior (21 LTS recomendado).
- Python 3.11.

Con ejecutar `bash setup.sh` en la raíz del repositorio, se comprueba la presencia de Docker y Java (guía la instalación si no los detecta), se crea el entorno virtual `.venv`, se instalan las dependencias de `requirements.txt` y se prepara la data en parquet si se encuentran las credenciales de Kaggle para descargar el dataset; caso contrario se presenta una guía para obtener y guardar dichas credenciales.

6.2 Modo automático de ejecución

El script `pipeline/tools/demo.py` orquesta el arranque del sistema: levanta el stack Docker, limpia las bases de datos, inicia el proceso Spark, espera a que las consultas hacia Kafka estén activas, y lanza el simulador reproduciendo los eventos más recientes de la data en los archivos parquet.

```
python pipeline/tools/demo.py           # publica 6 semanas de eventos
python pipeline/tools/demo.py --semanas 10 # publica 10 semanas de eventos
python pipeline/tools/demo.py --stop      # cierra los procesos y baja el stack
```

La opción `--semanas N` reproduce las últimas `N` semanas del subconjunto (valores disponibles entre 1 y 52; 6 por defecto). Debe tenerse en cuenta que la opción `--semanas N` cuenta las semanas desde la fecha más reciente hacia atrás, por lo que deben ajustarse los rangos de fechas en Grafana para cubrir las últimas `N` semanas.

Este script sirve para **demostrar** el sistema en funcionamiento, no para medir: las cifras de los KPIs se toman con el procedimiento manual de la siguiente sección.

6.3 Arranque manual (emisión de KPIs)

El arranque manual sigue cinco pasos, en este orden:

1. **Levantar el stack.**

```
docker compose -f pipeline/docker-compose.yml up -d
```

2. **Estado limpio.** Con el stack en marcha, borra los *checkpoints* de Spark, recrea los tópicos de Kafka y vacía las tablas de los sumideros.

```
python pipeline/tools/reset_state.py --yes
```

3. **Arrancar Spark.** El primer arranque descarga los conectores de Maven y tarda más de lo habitual; hay que esperar a que Spark informe de que las consultas hacia Kafka están en marcha antes de generar carga.

```
python pipeline/spark/stream_processing.py --trigger "1 second"
```

4. **Generar la carga.** El simulador reproduce la telemetría de los 652 sensores. El factor `--acelerar` aumenta la velocidad de publicación por segundo calculado como $652 \times (N / 3600)$, y `--limite` acota cuántos eventos se publicarán (sin `--limite` se reproducirán 5.682.185 eventos).

```
python pipeline/simulator/mqtt_simulator.py --acelerar 2000 --limite 50000
```

5. **Emitir el cuadro de KPIs.** `kpi_report.py` consulta los informes de actividad del pipeline y escribe el reporte en `pipeline/logs/informe_kpi.md`.

```
python pipeline/tools/kpi_report.py
```

El job de Spark debe estar procesando **antes** de que el simulador publique. Si se altera el orden, los tópicos en Kafka esperan a que aparezca quien los consuma y esa espera incrementa la latencia de persistencia, alterando la medición.

6.4 Escalado horizontal del bridge

La siguiente variación del paso 1 del arranque manual (sección 6.3) permite la replicación del bridge en N instancias:

```
docker compose -f pipeline/docker-compose.yml up -d --scale bridge=N
```

Mosquitto se encarga de repartir los mensajes entre las réplicas del bridge. El comando `failover_test.py --target bridge --fallo oom` provoca la caída de una sola, haciendo que las demás asuman la carga de la réplica caída. Una vez que `failover_test.py` repone la memoria por defecto, la política de reinicio en `docker-compose.yml` restablece el bridge a N replicas.

6.5 Herramientas de validación

Las mediciones de los KPIs se obtuvieron mediante tres scripts que residen en `pipeline/tools/`.

Herramienta	Función	Salida
<code>reset_state.py</code>	Deja el sistema en estado limpio: borra los <i>checkpoints</i> de Spark, recrea los tópicos de Kafka y vacía las tablas de los sumideros.	—
<code>kpi_report.py</code>	Interroga PostgreSQL, TimescaleDB, Apicurio y Grafana y emite el cuadro de los KPIs 1 a 5 en Markdown.	<code>pipeline/logs/informe_kpi.md</code>
<code>failover_test.py</code>	Lanza el simulador, provoca el fallo de un servicio, mide el tiempo de recuperación y la tasa de pérdida.	<code>pipeline/logs/ultimo_failover.json</code>

Tabla 6.1. Herramientas de validación de KPIs.

`kpi_report.py` mide el estado en que encuentra el pipeline para el instante en que es ejecutado. Para que sus cifras sean fiables, debe seguirse estrictamente el orden de la sección 6.3 (stack, estado limpio, Spark, simulador y reporte) antes de ser utilizado.

`failover_test.py` necesita que se haya ejecutado `reset_state.py` antes de iniciar el pipeline. El script verifica que el bridge y Spark estén en marcha antes de interrumpir algún servicio. Las opciones empleadas son:

- `failover_test.py --target mosquito --fallo kill`
- `failover_test.py --target kafka --fallo kill`
- `failover_test.py --target timescaledb --fallo oom`
- `failover_test.py --target postgres --fallo oom`
- `failover_test.py --target bridge --fallo oom`

Los servicios `mosquitto` y `kafka` no admiten `--fallo oom`: `Mosquitto` usa unos 3 MB, insuficiente para un fallo por memoria, y en `Kafka` un fallo de este tipo deja la JVM en bucle de reinicios.

6.5.1. Inyección de errores durante la publicación de eventos MQTT

El script `pipeline/tools/faulty_simulator.py` publica una mezcla de eventos reales y eventos inválidos para ejercitar la ruta de validación descrita en la sección 5.2.8 y dejar constancia de los rechazos en `iot.telemetry.dlq`.

```
python pipeline/tools/faulty_simulator.py # 10.000 eventos, 6 inválidos
python pipeline/tools/faulty_simulator.py --limite 20000 --fallas 300 # 20.000 eventos, 300 inválidos
```

`--limite` fija cuántos eventos reales se toman del dataset original (10.000 por defecto), y `--fallas` cuántos eventos inválidos se reparten a partes iguales entre seis motivos de rechazo por el bridge, en posiciones equiespaciadas dentro del lote.

Los seis motivos de rechazo introducidos son:

- `meter_reading` con valor infinito.
- `meter_reading` con valor NaN.
- `meter_reading` con valor no numérico.
- `building_id` ausente del evento.
- `building_id` como número en vez de cadena de texto.
- `timestamp` con valor nulo.

Al igual que con `mqtt_simulator.py`, cada hora de eventos del dataset es publicada en 1.8 segundos.

6.5.2. Visualización de los eventos rechazados por el bridge

Capítulo 7. DISCUSIÓN

7.1 Limitaciones del proyecto

Infraestructura de un solo nodo. El sistema se ha desplegado y validado sobre una pc personal mediante Docker Compose. Kafka se ejecuta como un solo nodo, por lo que el factor de replicación es necesariamente uno y no existen réplicas de las particiones a las que recurrir si el nodo cae; un despliegue real exigiría varios *brokers* para tener tolerancia efectiva a fallos. Además Spark Structured Streaming no se ejecuta como contenedor, sino desde el entorno virtual del *host* porque contenerizarlo exige recursos de procesador y memoria ya comprometidos con el resto de servicios.

Seguridad del entorno de desarrollo. El broker MQTT admite conexiones anónimas, lo cual es válido para el entorno local del proyecto pero se sustituiría en un despliegue real por autenticación con fichero de contraseñas o TLS mutuo.

Formato de los mensajes. El simulador publica cada evento como JSON en texto plano. Es la práctica recomendada para la fase de desarrollo, pero en producción se migraría a un formato binario como Sparkplug B, que es entre un 50 y un 80 % más pequeño que el JSON equivalente.

Descarte silencioso del broker bajo saturación. La cola de salida que Mosquitto mantiene por suscriptor está limitada a 10.000 mensajes. Si el bridge confirma los mensajes más despacio de lo que le llegan durante un tiempo sostenido y esa cola se llena, el broker descarta los mensajes siguientes sin emitir ningún error.

Aproximaciones en la agregación por ventana. La columna `distinct_buildings` de las métricas agregadas es un recuento aproximado de edificios, porque Spark Structured Streaming no admite el conteo exacto de valores distintos en agregaciones de flujo.

Alcance de la evaluación. Todas las mediciones se obtuvieron con el simulador MQTT acelerado y no con el ritmo real del parque de medidores; reproducir la cadencia horaria real habría exigido semanas de ejecución continua para obtener comprobación de la correcta ingesta, validación y persistencia de los datos procesados.

7.2 Impacto del resultado

El impacto directo del proyecto es servir de implementación de referencia allí donde la revisión del estado del arte encontró un vacío. Esa revisión puso de manifiesto que, si bien existen numerosos casos ilustrativos de arquitecturas Kappa en IoT, ninguna de las revisadas combina los tres elementos que este proyecto integra en código abierto: un broker MQTT ligero acoplado a un microservicio de *bridging* hacia Kafka, un registro de esquemas Avro que valida el formato y la admisibilidad de cada evento, y una estrategia de doble sumidero que separa el consumo operacional del analítico.

Más allá del caso de uso energético, el diseño implementado es transferible a otros escenarios de monitorización sobre grandes volúmenes de datos de sensores: gestión de edificios inteligentes, instrumentación industrial o cualquier dominio en el que convivan la necesidad de vigilancia inmediata y la de análisis histórico. Al estar construido con herramientas de código abierto, containerizado y disponible desde el repositorio, el sistema puede adoptarse como punto de partida por terceros, cumpliendo así el objetivo de ser una referencia técnica reutilizable.

Capítulo 8. CONCLUSIONES

8.1 Conclusiones del trabajo

El alcance del proyecto: diseñar, desarrollar y validar un sistema escalable de microservicios para la ingesta, el procesamiento y el análisis de datos IoT, aplicado a la supervisión en tiempo real de la telemetría de consumo energético, fue cumplido en su totalidad.

Se probó la madurez y capacidad de las herramientas de código abierto para implementar un sistema de tratamiento de datos en flujo, resiliente y escalable, presentándolo como alternativa ante la amplia oferta de herramientas gestionadas y sistemas en la nube.

Con la debida consideración de las limitaciones en este proyecto, se lograron cumplir los diversos objetivos a lo largo de su desarrollo:

1. **Persistencia de telemetría rápida y fiable.** El p95 de la telemetría validada se persistió en un tiempo inferior a los 25 segundos, con 0 % de mensajes perdidos durante el procesamiento.
2. **Gobernanza del esquema de datos.** El 100 % de los eventos ingeridos se validaron contra el esquema Avro registrado, sin fallos de deserialización ni mensajes erróneamente desviados al tópico de inválidos.
3. **Procesamiento en flujo con baja latencia.** Los micro-lotes de Spark Structured Streaming se procesaron por debajo de los 2 segundos, a una tasa estable sin incremento de tiempo.
4. **Visualización diferenciada operacional y analítica.** Se diseñaron e implementaron los dashboards de Grafana y las páginas de Power BI con las visualizaciones del consumo energético, la eficiencia operativa, la detección de anomalías y el análisis histórico.
5. **Resiliencia del sistema.** Se comprobó la capacidad de auto recuperación de los componentes del sistema en menos de 60 segundos con 0 % de pérdida de datos.

Finalmente, la containerización, documentación y disposición del proyecto al público, lo hace disponible para su adopción, mejoramiento y adaptación a otros proyectos similares de monitorización energética.

Capítulo 9. FUTURAS LÍNEAS DE TRABAJO

La implementación actual del pipeline se ha desplegado íntegramente mediante Docker Compose, lo cual resulta adecuado para el alcance de este TFM (un entorno de desarrollo y validación en una única máquina), pero presenta limitaciones evidentes de cara a un despliegue productivo real: no ofrece escalado horizontal automático de los componentes con mayor carga (particiones de Kafka, ejecutores de Spark Structured Streaming), ni mecanismos nativos de auto-recuperación ante fallos de nodo.

Una línea de evolución natural del sistema es la migración de la orquestación hacia una distribución ligera de Kubernetes, manteniendo el resto de la arquitectura (Mosquitto, Kafka en modo KRaft, Apicurio, Spark Structured Streaming y los sinks duales TimescaleDB/PostgreSQL) sin cambios sustanciales, ya que todos los componentes están containerizados y son, en principio, portables a manifiestos de Kubernetes o a Helm charts.

Entre las distribuciones evaluadas como candidatas destacan dos por su bajo consumo de recursos y su idoneidad para escenarios de IoT y edge computing:

- **K3s** (Rancher/SUSE): una distribución certificada por la CNCF (Cloud Native Computing Foundation), lo que garantiza compatibilidad completa con la API estándar de Kubernetes, que sustituye *etcd* por SQLite en despliegues de un solo nodo y reduce drásticamente el consumo de memoria frente a Kubernetes estándar, lo que la convierte en una de las opciones más utilizadas en estudios comparativos de orquestación de contenedores para entornos de recursos limitados.
- **MicroK8s** (Canonical): distribución empaquetada como snap, con fuerte integración en el ecosistema Ubuntu (el mismo utilizado en el entorno de desarrollo de este proyecto) y un sistema de complementos (*addons*) que simplifica la activación de componentes como almacenamiento persistente o ingress.

Como trabajo futuro, se propone:

1. Migrar el stack de Docker Compose a manifiestos de Kubernetes (o Helm charts), comenzando por un clúster de un solo nodo sobre K3s o MicroK8s en el mismo entorno WSL/Ubuntu ya utilizado para el desarrollo.
2. Evaluar el escalado horizontal de los ejecutores de Spark Structured Streaming y de las particiones de Kafka bajo carga variable, midiendo consumo de CPU, memoria y latencia, replicando la metodología de estudios recientes que han comparado el rendimiento y la eficiencia de recursos de varias distribuciones ligeras de Kubernetes en escenarios de edge computing bajo esas mismas métricas [21] y de trabajos que han evaluado herramientas de orquestación de contenedores específicamente en dichos entornos [22].
3. Explorar la incorporación de mecanismos de auto-recuperación y despliegue continuo (GitOps) sobre el clúster resultante, como extensión natural del repositorio actual del proyecto.

Bibliografía

- [1] Y. Liu, «Data Analytics in the Internet of Things Era: Tools, Approaches, Challenges, and Solutions,» *Journal of ICT Standardization*, vol. 13, n.º 4, págs. 427-448, 2025. DOI: 10.13052/jicts2245-800X.1344.
- [2] N. Marz, *How to Beat the CAP Theorem*, Thoughts from the Red Planet blog post, oct. de 2011. dirección: <https://nathanmarz.com/blog/how-to-beat-the-cap-theorem.html>.
- [3] N. Marz y J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. 20 Baldwin Road, PO Box 761, Shelter Island, NY 11964, USA: Manning Publications Co., 2015, pág. 328, ISBN: 9781617290343.
- [4] J. Kreps, *Questioning the Lambda Architecture*, O'Reilly Radar blog post, jul. de 2014. dirección: <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>.
- [5] H. Cao y M. Wachowicz, «An Edge-Fog-Cloud Architecture of Streaming Analytics for Internet of Things Applications,» *Sensors*, vol. 19, n.º 16, pág. 3594, 2019. DOI: 10.3390/s19163594.
- [6] S. Shinde, *Kappa Architecture Unveiled: Real-Time IoT Innovation and Hands-On Implementation*, Level Up Coding blog post, mar. de 2025. dirección: <https://levelup.gitconnected.com/kappa-architecture-unveiled-real-time-iot-innovation-and-hands-on-implementation-d8ff57f37167>.
- [7] G. Zefkilis, *Kappa Architecture in Action: From Sensors to Dashboards with Kafka, Spark Streaming, StarRocks, MinIO, and Docker*, Data Engineer Things blog post, oct. de 2025. dirección: <https://blog.dataengineerthings.org/kappa-architecture-in-action-from-sensors-to-dashboards-with-kafka-spark-streaming-starrocks-80492a509e1a>.
- [8] EMQ Technologies, *Data Integration*, Documentación oficial de EMQX Enterprise, 2026. dirección: <https://docs.emqx.com/en/emqx/latest/data-integration/data-bridges.html>.
- [9] HiveMQ, *HiveMQ Enterprise Extension for Kafka*, Documentación oficial de producto, 2026. dirección: <https://docs.hivemq.com/hivemq-kafka-extension/latest/index.html>.
- [10] Lenses.io, *Stream Reactor: Kafka Connect connectors*, Repositorio del proyecto, licencia Apache 2.0, 2026. dirección: <https://github.com/lensesio/stream-reactor>.
- [11] S. Lakshminpathy, *Apache Flink vs Apache Kafka Streams vs Apache Spark Structured Streaming: Comparing Stream Processing Engines*, Onehouse blog post, abr. de 2025. dirección: <https://www.onehouse.ai/blog/apache-spark-structured-streaming-vs-apache-flink-vs-apache-kafka-streams-comparing-stream-processing-engines>.

- [12] InfluxData, *InfluxDB 3.0 vs OSS Benchmarks*, Documento oficial de benchmarking, ago. de 2023. dirección: <https://get.influxdata.com/rs/972-GDU-533/images/InfluxDB-3-0-vs-OSS-Benchmarks.pdf>.
- [13] TigerData, *TimescaleDB vs. InfluxDB*, TigerData blog post, jul. de 2023. dirección: <https://www.tigerdata.com/blog/timescaledb-vs-influxdb-for-time-series-data-timescale-influx-sql-nosql-36489299877>.
- [14] QuestDB, *InfluxDB 3 Core Benchmarks: QuestDB Comparison*, QuestDB blog post, ago. de 2026. dirección: <https://questdb.com/blog/influxdb3-core-benchmarks/>.
- [15] T. Domínguez-Bolaño, O. Campos, V. Barral, C. J. Escudero y J. A. García-Naya, «An overview of IoT architectures, technologies, and existing open-source projects,» *Internet of Things*, 2022. DOI: 10.1016/j.iot.2022.100626.
- [16] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. 1005 Gravenstein Highway North, Sebastopol, CA 95472, USA: O'Reilly Media, 2017, cap. 4, ISBN: 9781449373320.
- [17] D. Kim, S. Park, Y. Park, V. Falconer e Y. Yun, *How SOCAR Built a Streaming Data Pipeline to Process IoT Data for Real-Time Analytics and Control*, AWS blog post, nov. de 2022. dirección: <https://aws.amazon.com/blogs/big-data/how-socar-built-a-streaming-data-pipeline-to-process-iot-data-for-real-time-analytics-and-control/>.
- [18] ASHRAE, *ASHRAE - Great Energy Predictor III*, Competición de Kaggle, 2019. dirección: <https://www.kaggle.com/competitions/ashrae-energy-prediction>.
- [19] G. Hohpe y B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003, pág. 736, ISBN: 9780321200686.
- [20] R. Yokota, *Best Practices for Confluent Schema Registry*, Confluent blog post, mar. de 2024. dirección: <https://www.confluent.io/blog/best-practices-for-confluent-schema-registry/>.
- [21] H. Koziolok y N. Eskandani, «Lightweight Kubernetes Distributions: A Performance Comparison of MicroK8s, K3s, K0s, and Microshift,» en *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE 2023)*, New York, NY, USA: ACM, 2023, págs. 17-29. DOI: 10.1145/3578244.3583737.
- [22] I. Čilić, P. Krivić, I. Podnar Žarko y M. Kušek, «Performance Evaluation of Container Orchestration Tools in Edge Computing Environments,» *Sensors*, vol. 23, n.º 8, pág. 4008, 2023. DOI: 10.3390/s23084008.

Capítulo 10. ANEXOS

Sirven para incluir documentación complementaria (planos, circuitos, código, ficheros de configuración, especificaciones técnicas y hojas de características, fichas explicativas, resultado de encuestas, reglamentación y normativas requeridas, etc.).